GRUNDLAGEN ALGORITHMISCHER PROBLEMLÖSUNG
- PROBLEMZERLEGUNG -

BIRGITTA KÖNIG-RIES

HEINZ-NIXDORF-PROFESSUR FÜR VERTEILTE INFORMATIONSSYSTEME
UNIVERSITÄT JENA

PAD FÜR DIE VERANSTALTUNG

https://pad.uni-jena.de/wYY3gc6PR-aBbMknOXpL8A

ÜBERSICHT

Was bisher geschah

Übersicht: Ansätze zur Zerlegung von Problemen

Beispielansätze im Detail

Who‘s Who

Beispielansätze im Detail

Zusammenfassung und Ausblick

ÜBERSICHT

Was bisher geschah

Übersicht: Ansätze zur Zerlegung von Problemen

Beispielansätze im Detail

Who‘s Who

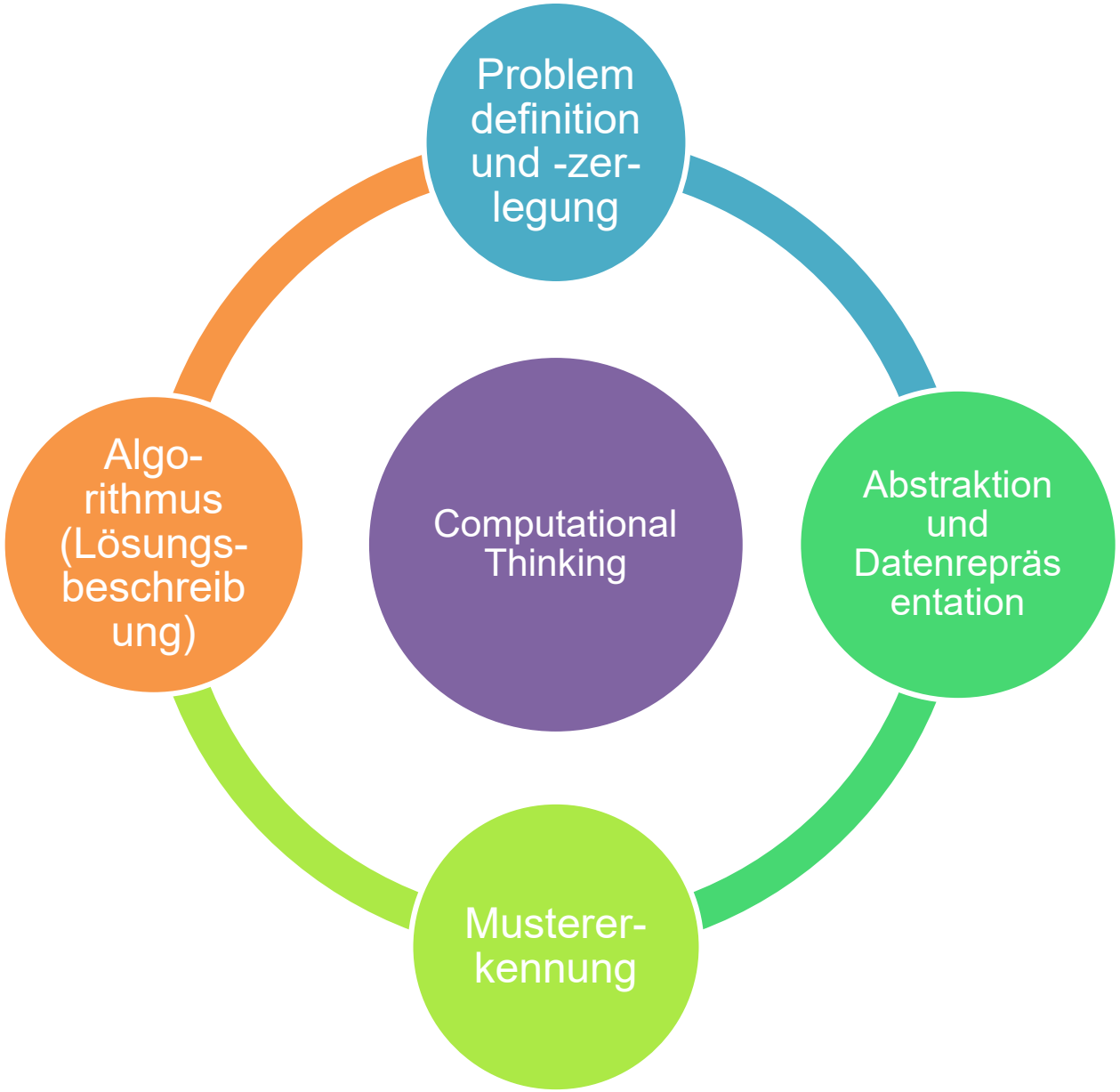
Beispielansätze im Detail

Zusammenfassung und Ausblick

DIE VERANSTALTUNG IM ÜBERBLICK

Phase	Fokus	Leitfragen
I. Problemanalyse & Repräsentation (T1–T4)	Wie erfasse und strukturiere ich ein Problem?	Was sind die relevanten Daten und Prozesse?
II. Modellierung & Abstraktion (T5–T7)	Wie bilde ich Realität in Modellen ab?	Welche Strukturen, Beziehungen, Zustände gibt es?
III. Algorithmisches Denken & Qualität (T8–T11)	Wie werden Modelle zu effizienten, korrekten Verfahren?	Wie messe ich Qualität und teste Lösungen?
IV. Zusammenarbeit & Reflexion (T12–T15)	Wie entsteht daraus gute Software im Team?	Wie organisieren, verbessern, kommunizieren wir?

COMPUTATIONAL THINKING (2)



TERMIN 2: PROBLEME STRUKTURIEREN UND ZERLEGEN

- Dekomposition als zentrales Prinzip der Informatik
- Zusammenhang zwischen Dekomposition und Wiederverwendung



ÜBERSICHT

Was bisher geschah

Übersicht: Ansätze zur Zerlegung von Problemen

Beispielansätze im Detail

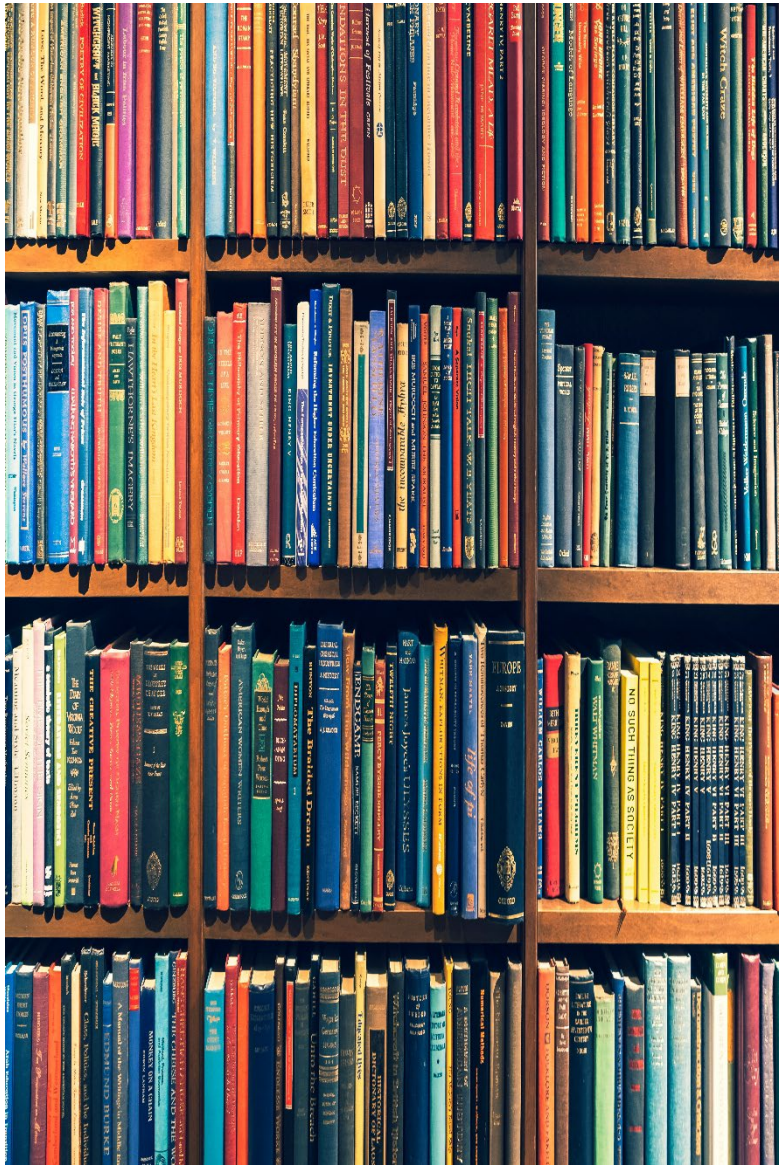
Who‘s Who

Beispielansätze im Detail

Zusammenfassung und Ausblick

BEISPIELPROBLEME AUS DEM ALLTAG







ANSÄTZE ZUR PROBLEMZERLEGUNG?

https://pad.uni-jena.de/wYY3gc6PR-aBbMknOXpL8A

ARTEN DER PROBLEMZERLEGUNG

(kein Anspruch auf Vollständigkeit!)

- Funktionelle Modularisierung
- Schichtenarchitekturen
- Rekursive Zerlegung
- Parallelisierung

ÜBERSICHT

Was bisher geschah

Übersicht: Ansätze zur Zerlegung von Problemen

Beispielansätze im Detail

Who‘s Who

Beispielansätze im Detail

Zusammenfassung und Ausblick

HIERARCHISCHE ZERLEGUNG: SCHICHTENARCHITEKTUREN

KOMMUNIKATION IN COMPUTERNETZWERKEN

Welche Probleme müssen gelöst werden, damit Ihr Rechner auf die Webseite der Uni zugreifen kann?

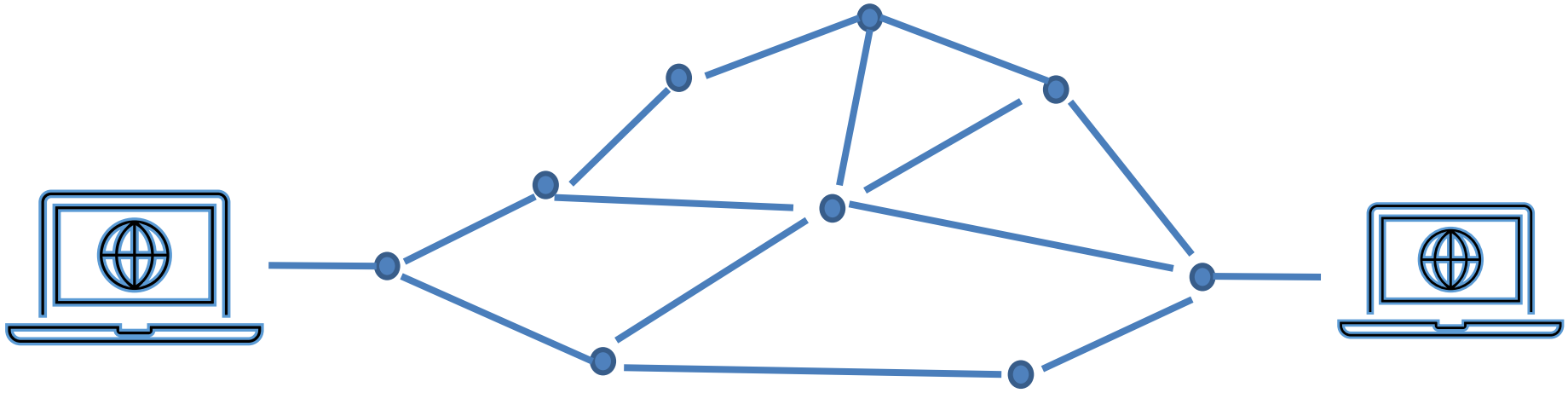
https://pad.uni-jena.de/wYY3gc6PR-aBbMknOXpL8A

KOMMUNIKATION IN COMPUTERNETZWERKEN

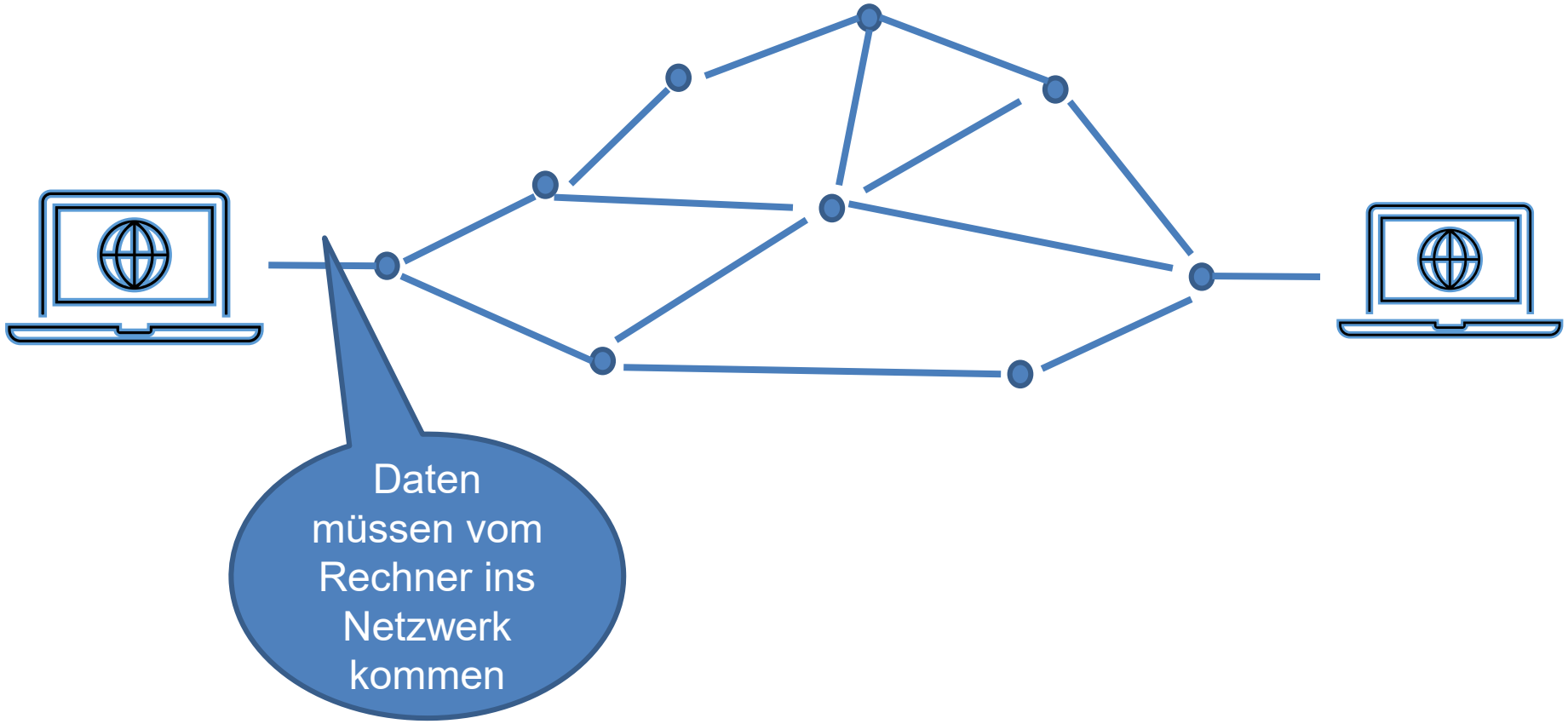




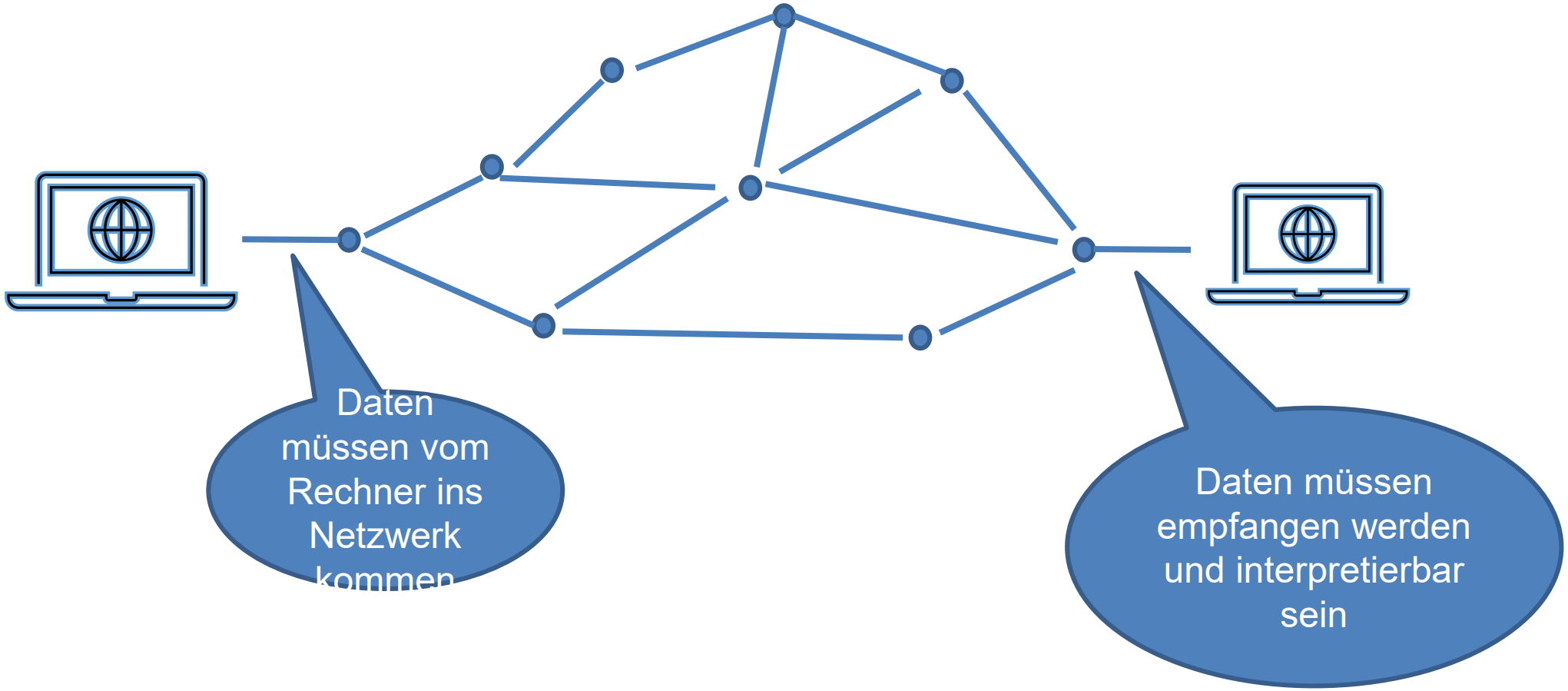
KOMMUNIKATION IN COMPUTERNETZWERKEN



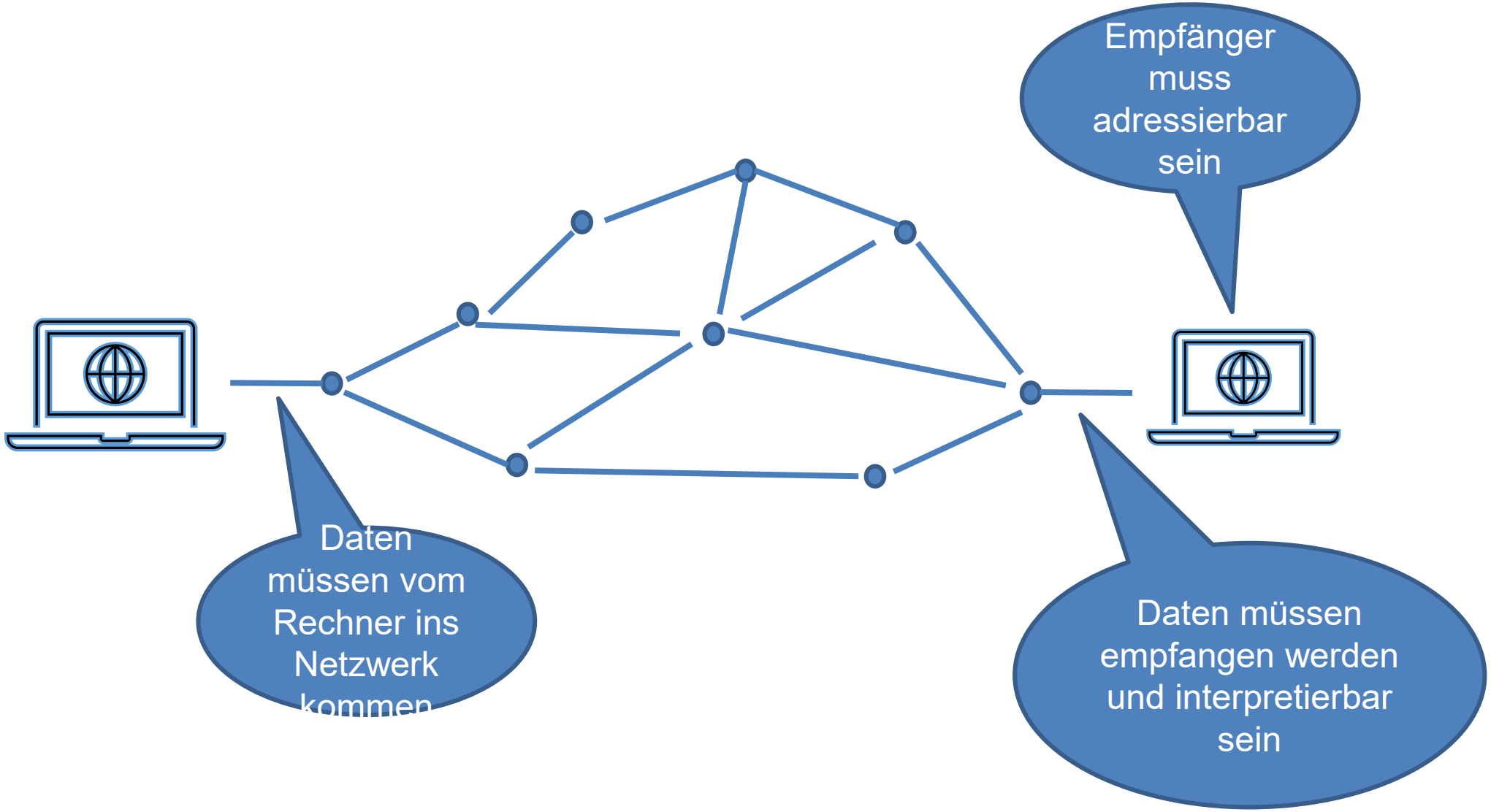
KOMMUNIKATION IN COMPUTERNETZWERKEN



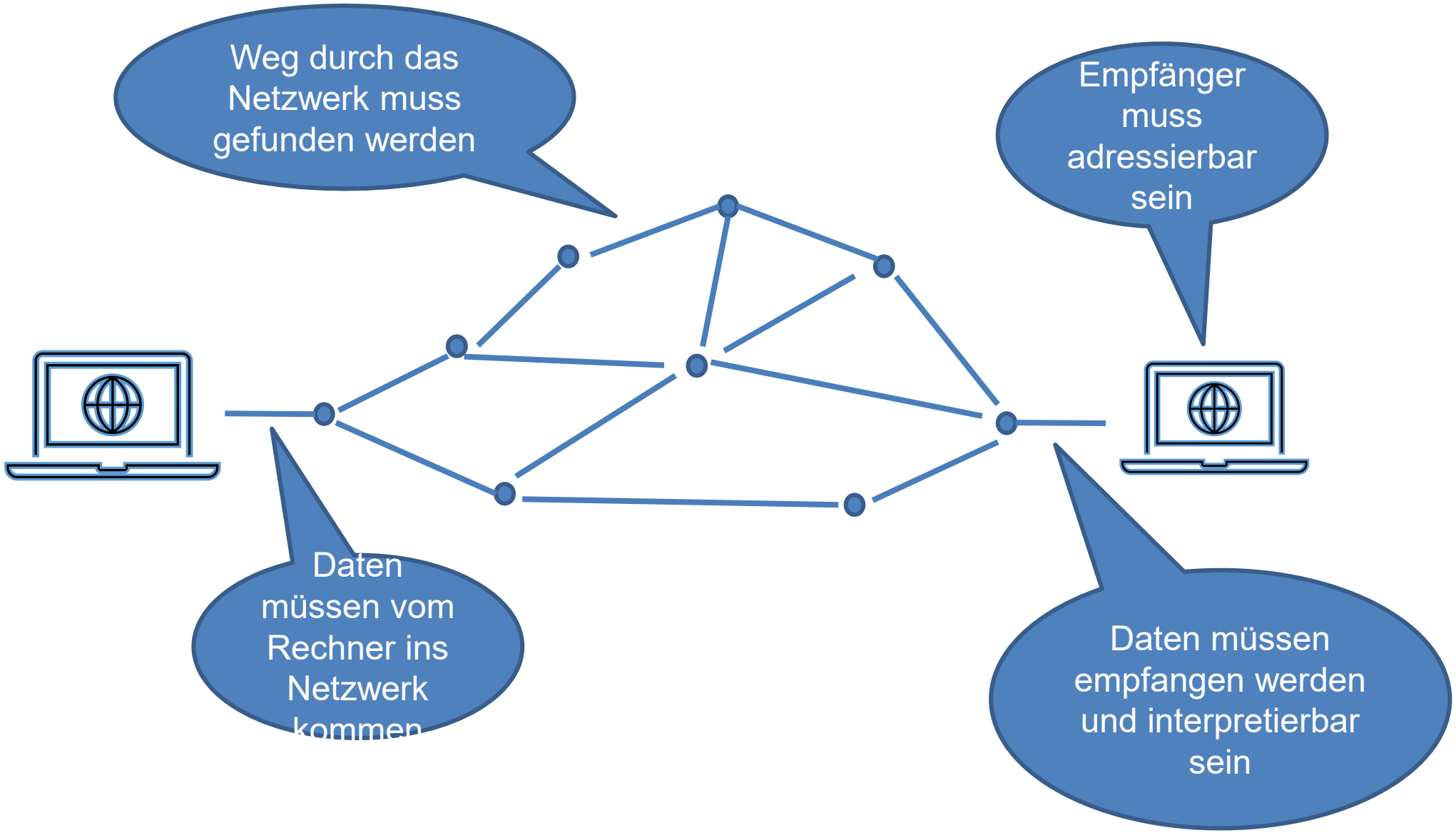
KOMMUNIKATION IN COMPUTERNETZWERKEN



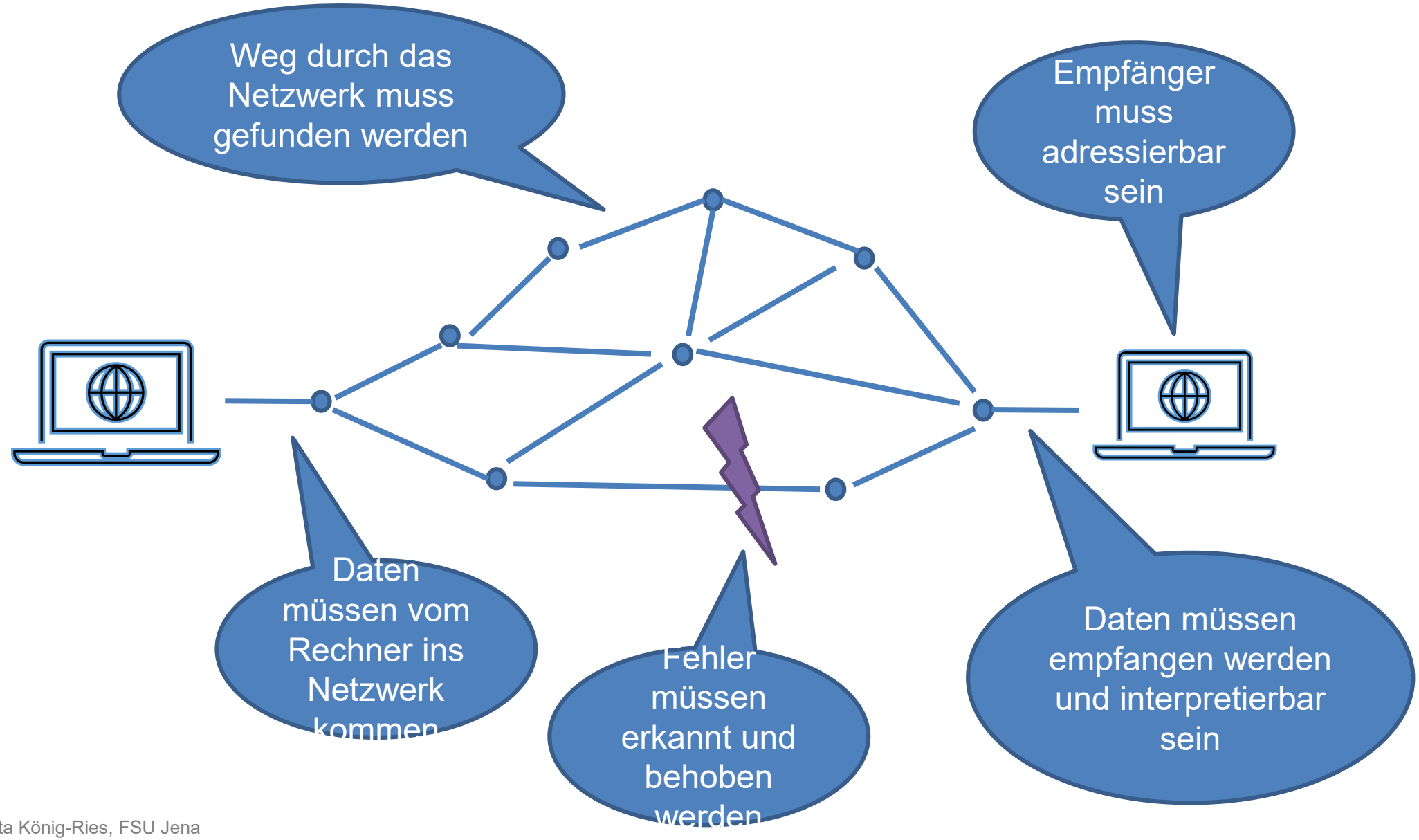
KOMMUNIKATION IN COMPUTERNETZWERKEN



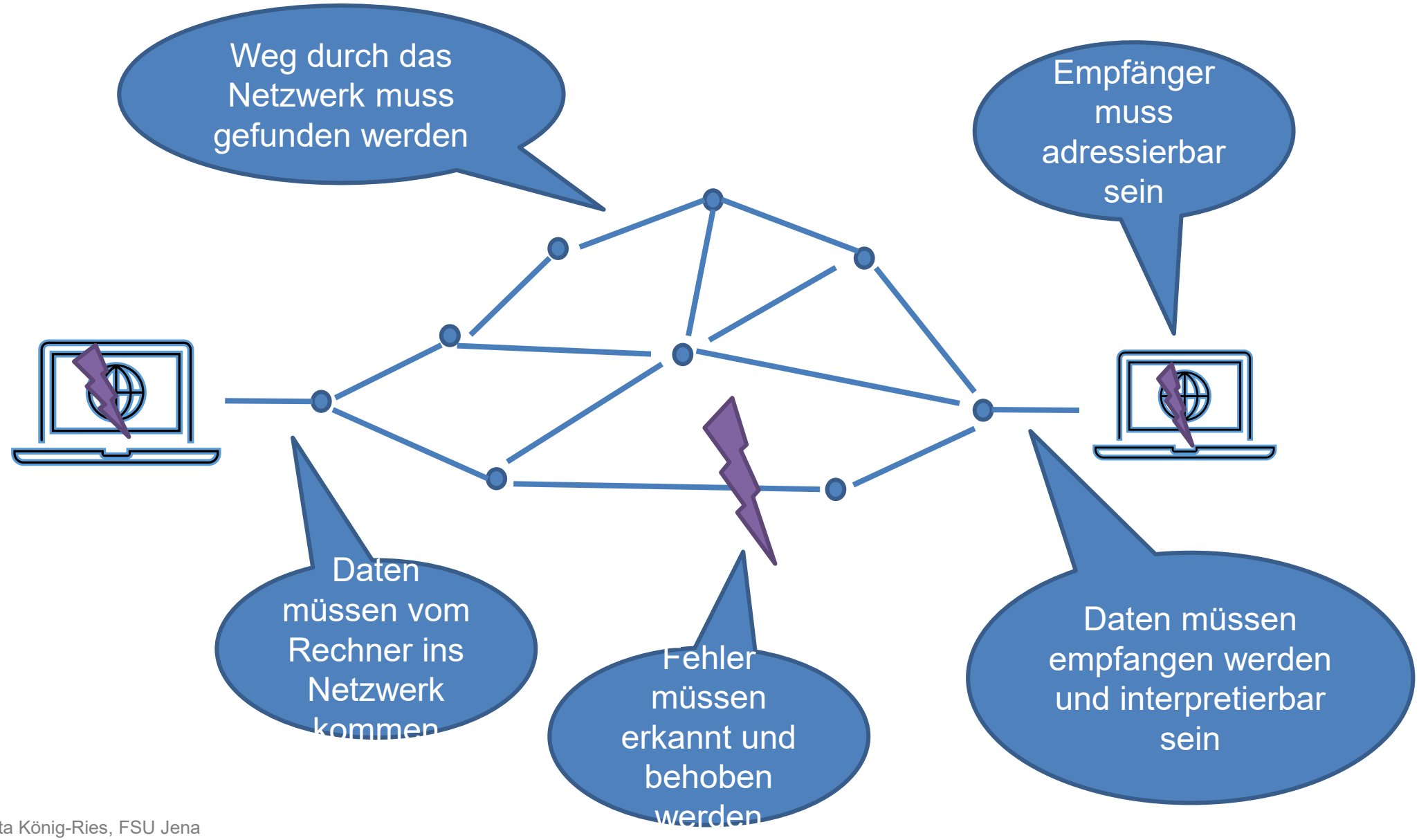
KOMMUNIKATION IN COMPUTERNETZWERKEN



KOMMUNIKATION IN COMPUTERNETZWERKEN



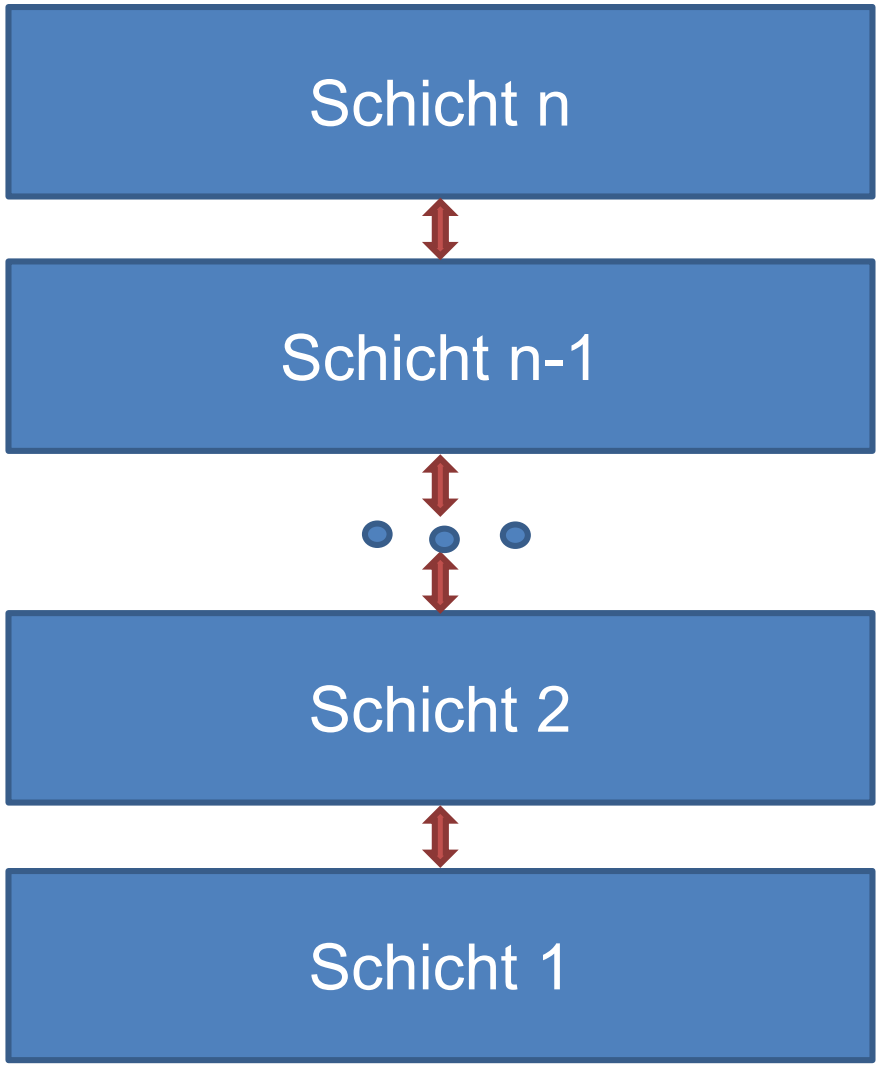
KOMMUNIKATION IN COMPUTERNETZWERKEN



KOMMUNIKATION IN COMPUTERNETZWERKEN

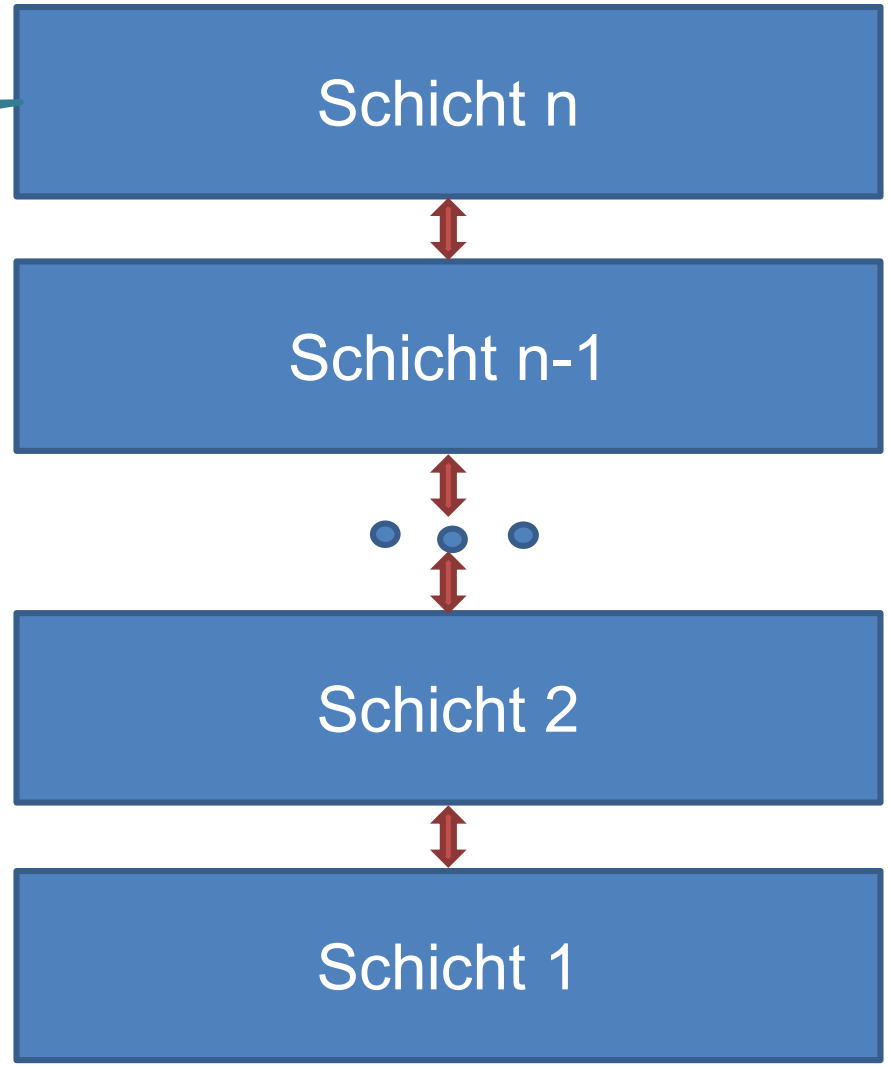


SCHICHTENARCHITEKTUREN

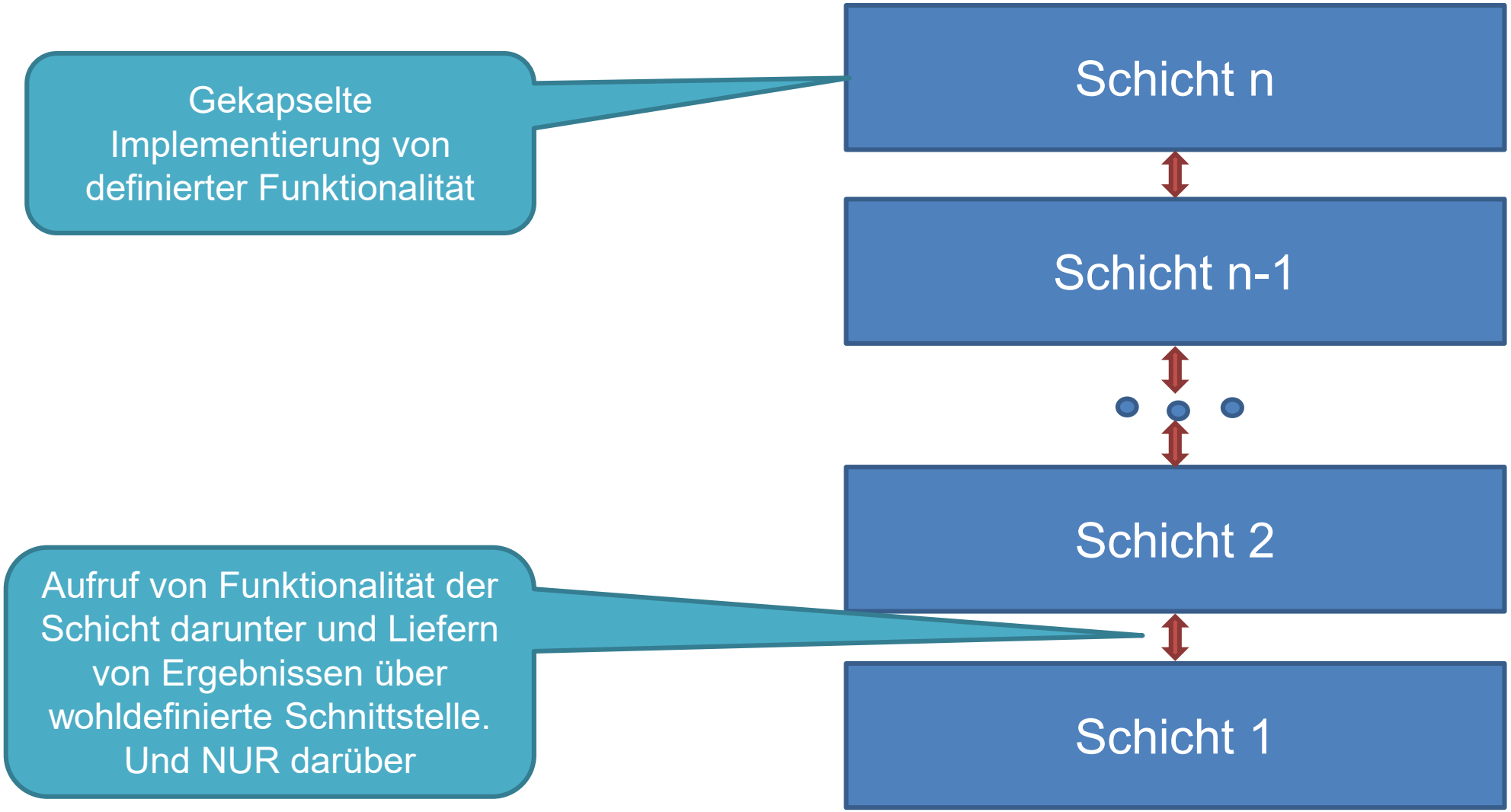


SCHICHTENARCHITEKTUREN

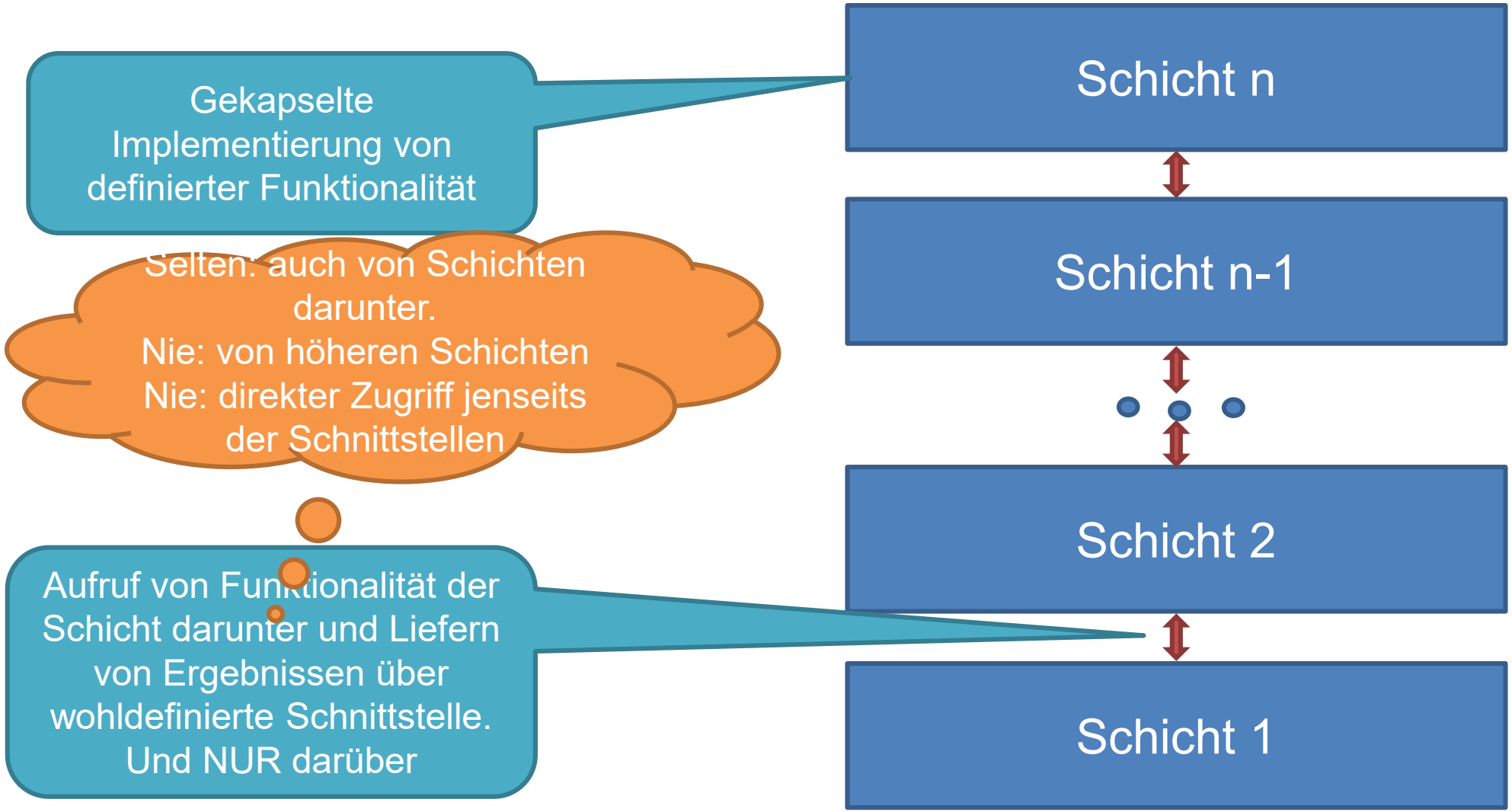
Gekapselte Implementierung von definierter Funktionalität



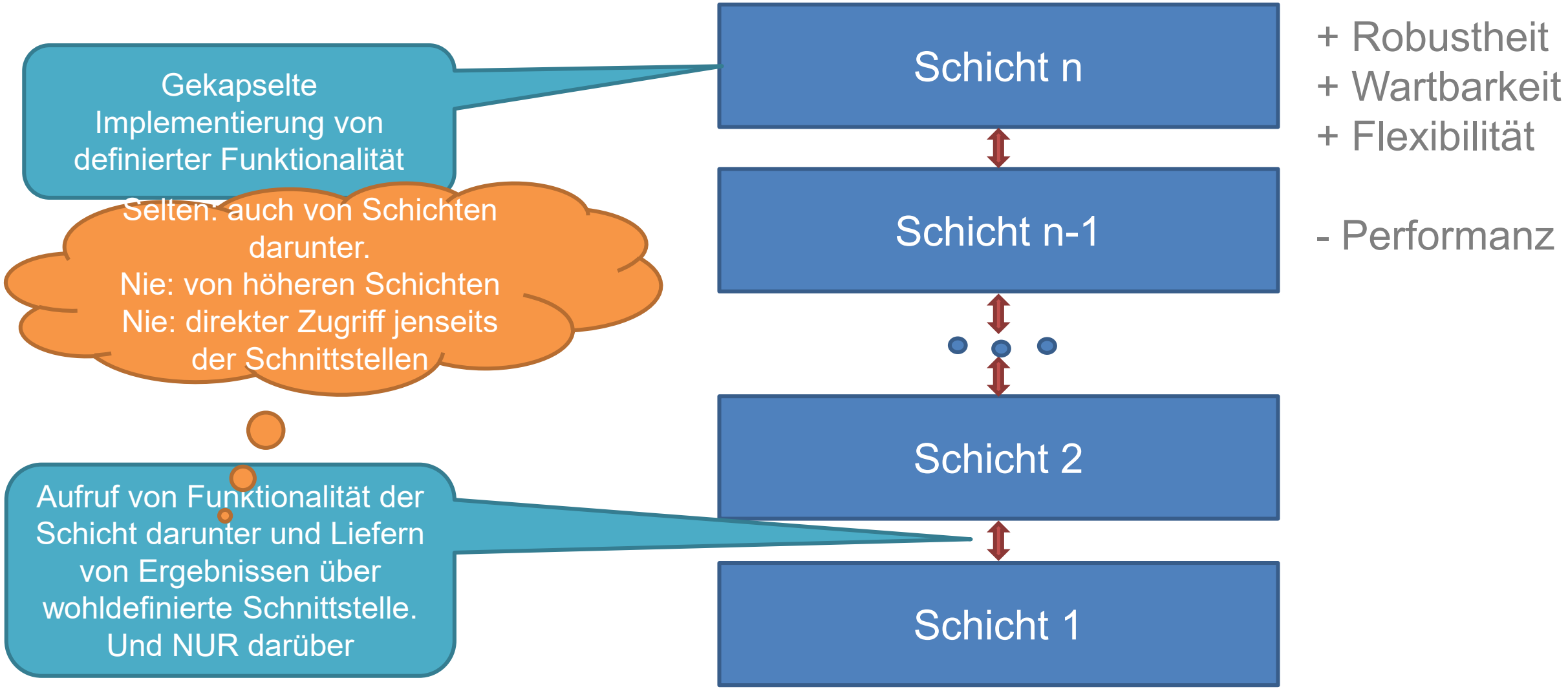
SCHICHTENARCHITEKTUREN



SCHICHTENARCHITEKTUREN



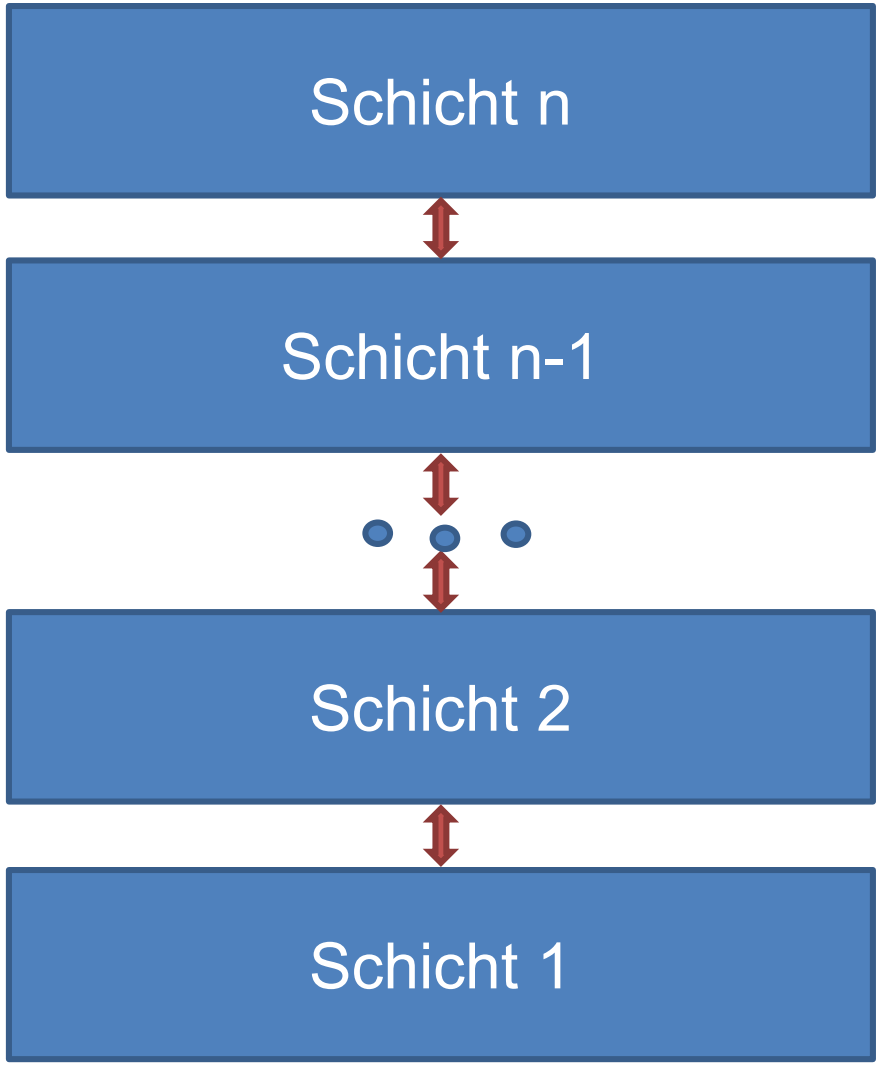
SCHICHTENARCHITEKTUREN

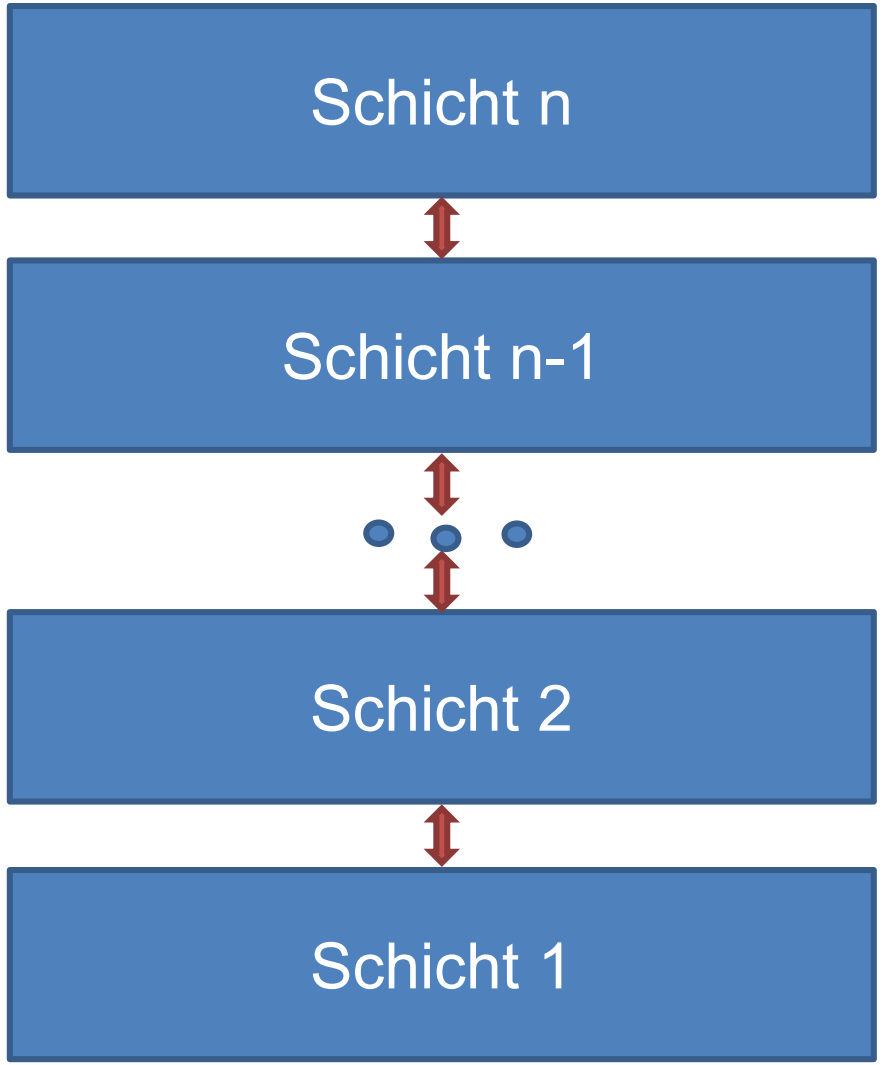


+ Robustheit
+ Wartbarkeit
+ Flexibilität

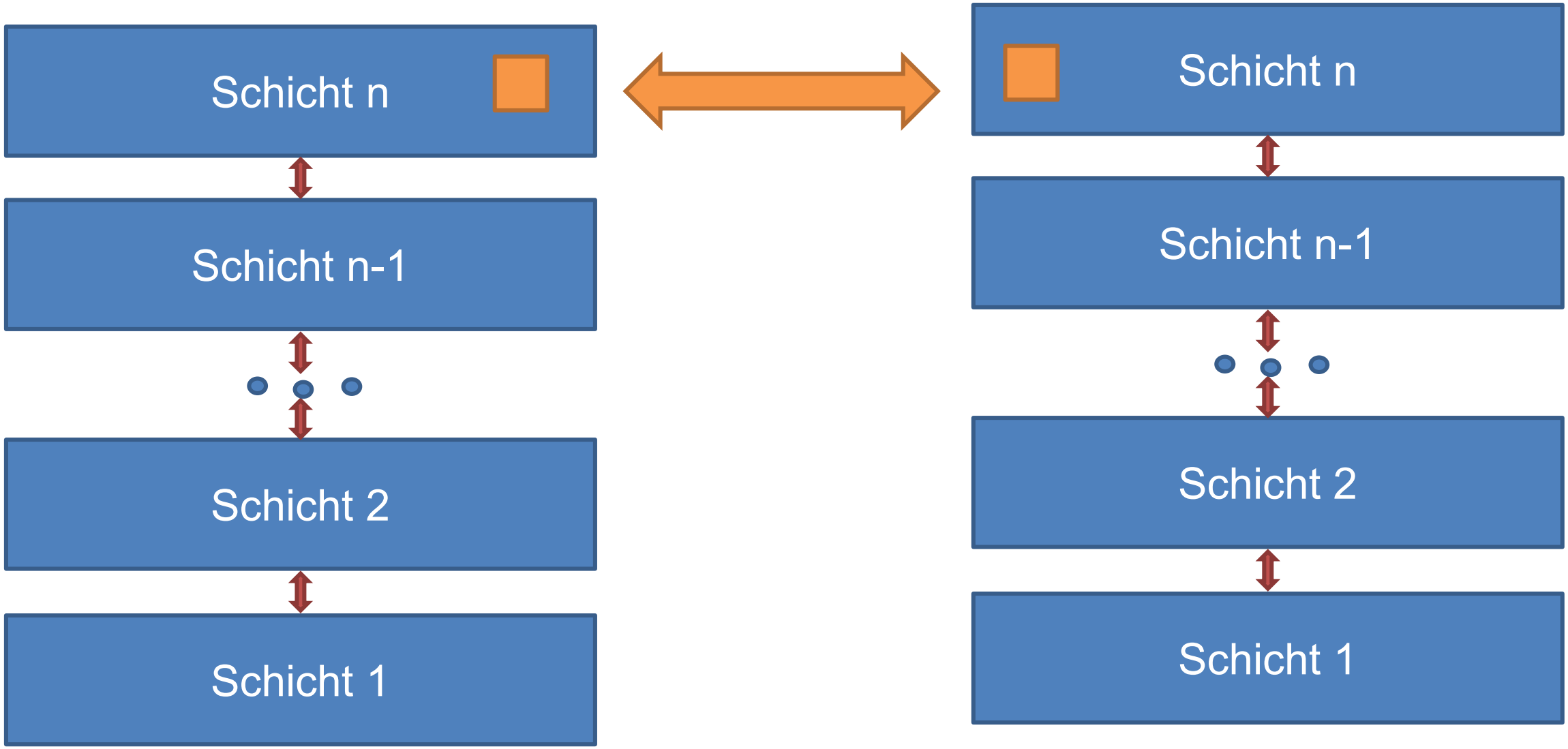
- Performanz

SCHICHTENARCHITEKTUREN

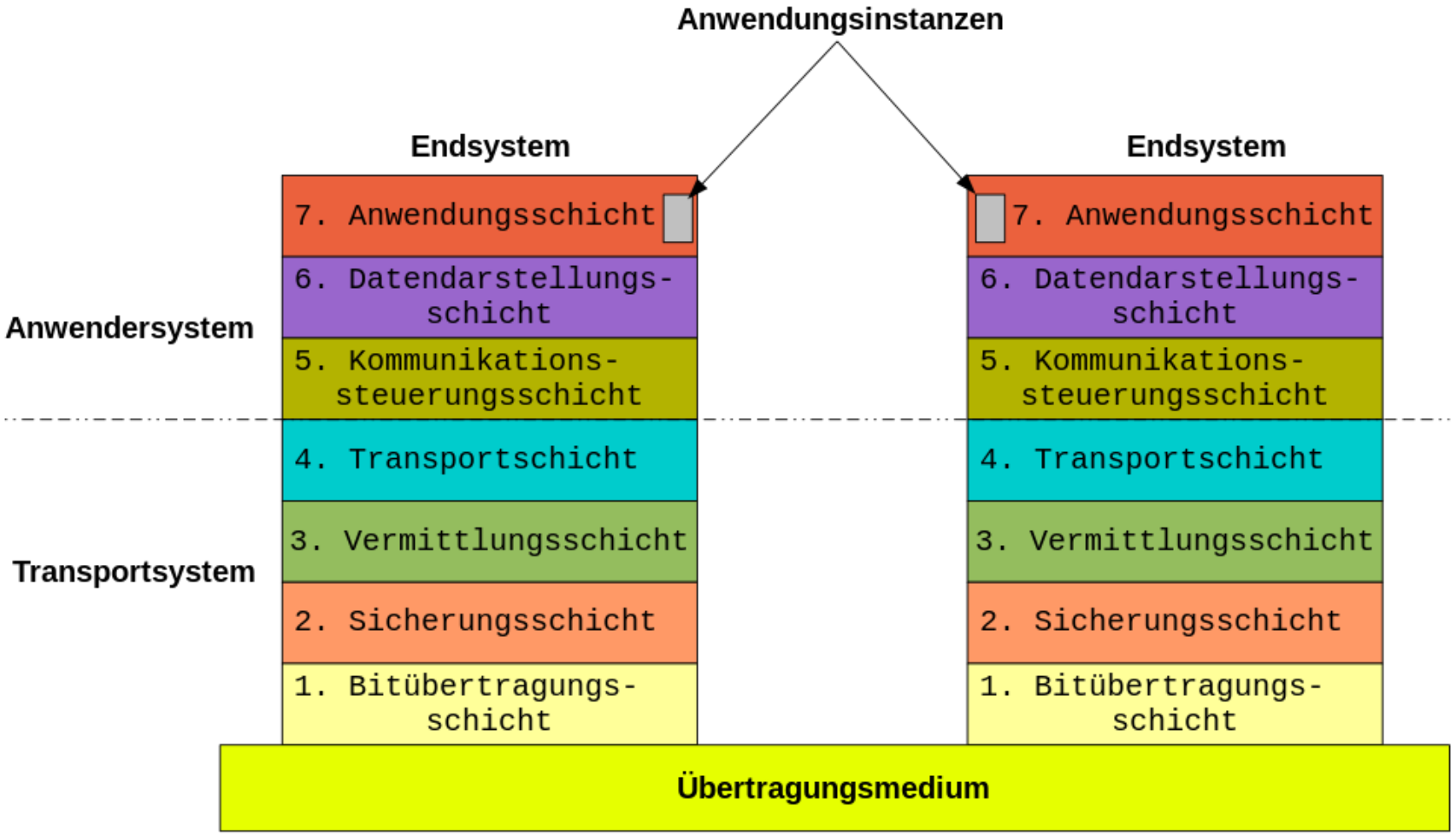




SCHICHTENARCHITEKTUREN



ISO/OSI-REFERENZMODELL

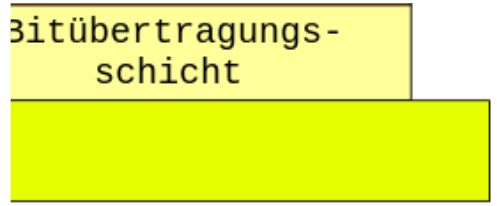


Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

ISO/OSI-REFERENZMODELL

Schnittstelle zwischen Rechner und physikalischem Medium
Drahtgebunden oder drahtlos

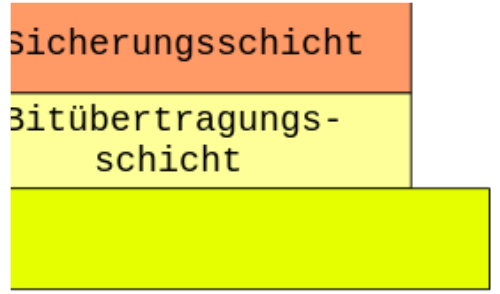
Kodierung von Bitströmen
Keine Fehlererkennung möglich



Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

ISO/OSI-REFERENZMODELL

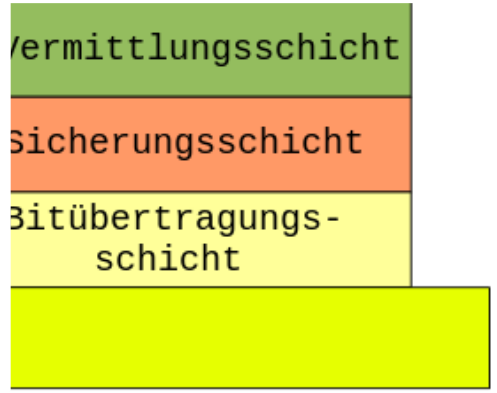
Aufbau eines gesicherten Kanals
Zerlegung des Bitstroms in Blöcke (frames) mit Prüfsummen
Datenflusskontrolle
Regelung des Zugangs auf das Übertragungsmedium



Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

ISO/OSI-REFERENZMODELL

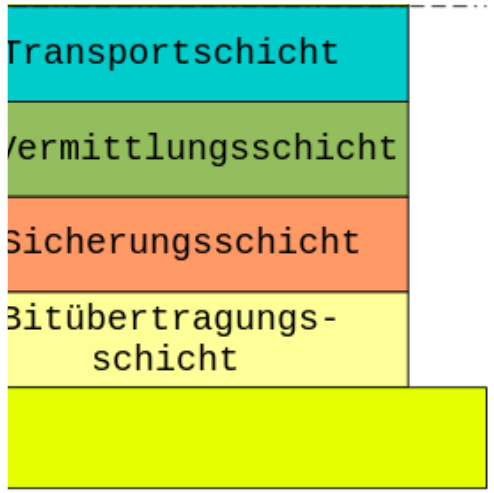
Netzwerkübergreifende Adressierung
Wegewahl
Leitungs- oder paketorientiert
Ggf. Zerlegung in Pakete



Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

ISO/OSI-REFERENZMODELL

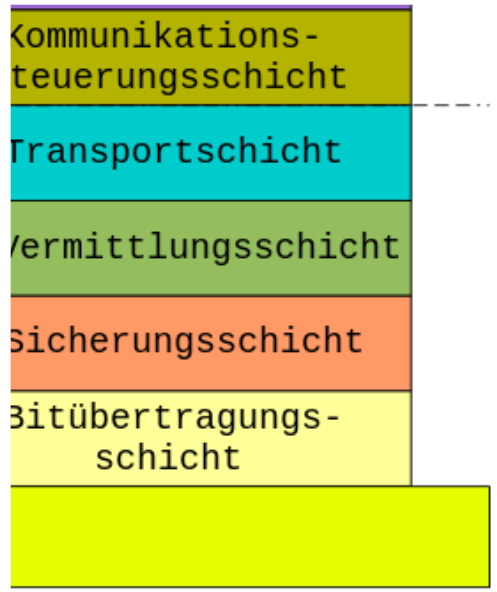
Stauvermeidung
Übertragung mit (oder ohne) Garantien



Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

ISO/OSI-REFERENZMODELL

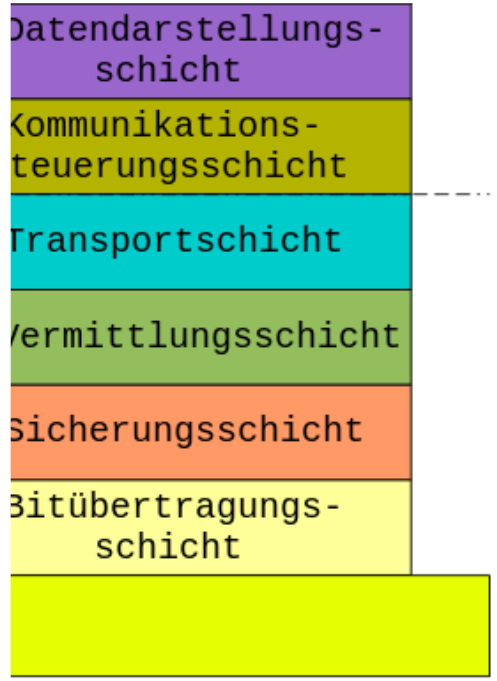
Auch: Sitzungsschicht
Gruppierung von Nachrichten nach Anwendungsgesichtspunkten



Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

ISO/OSI-REFERENZMODELL

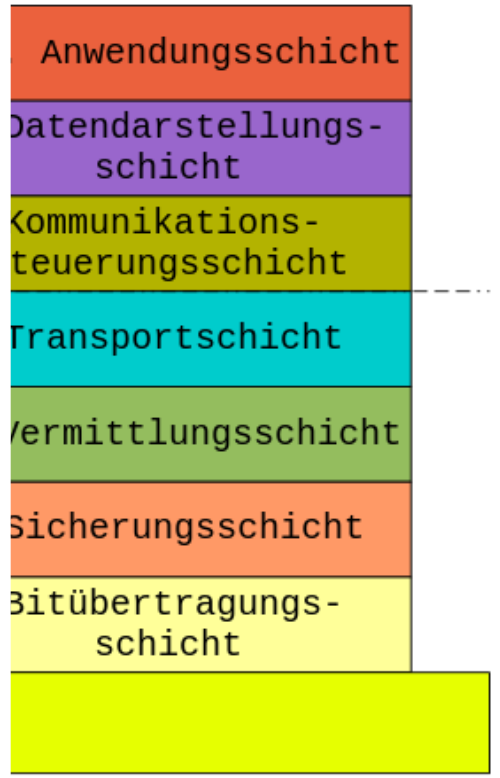
Anpassung unterschiedlicher Datenrepräsentationen



Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

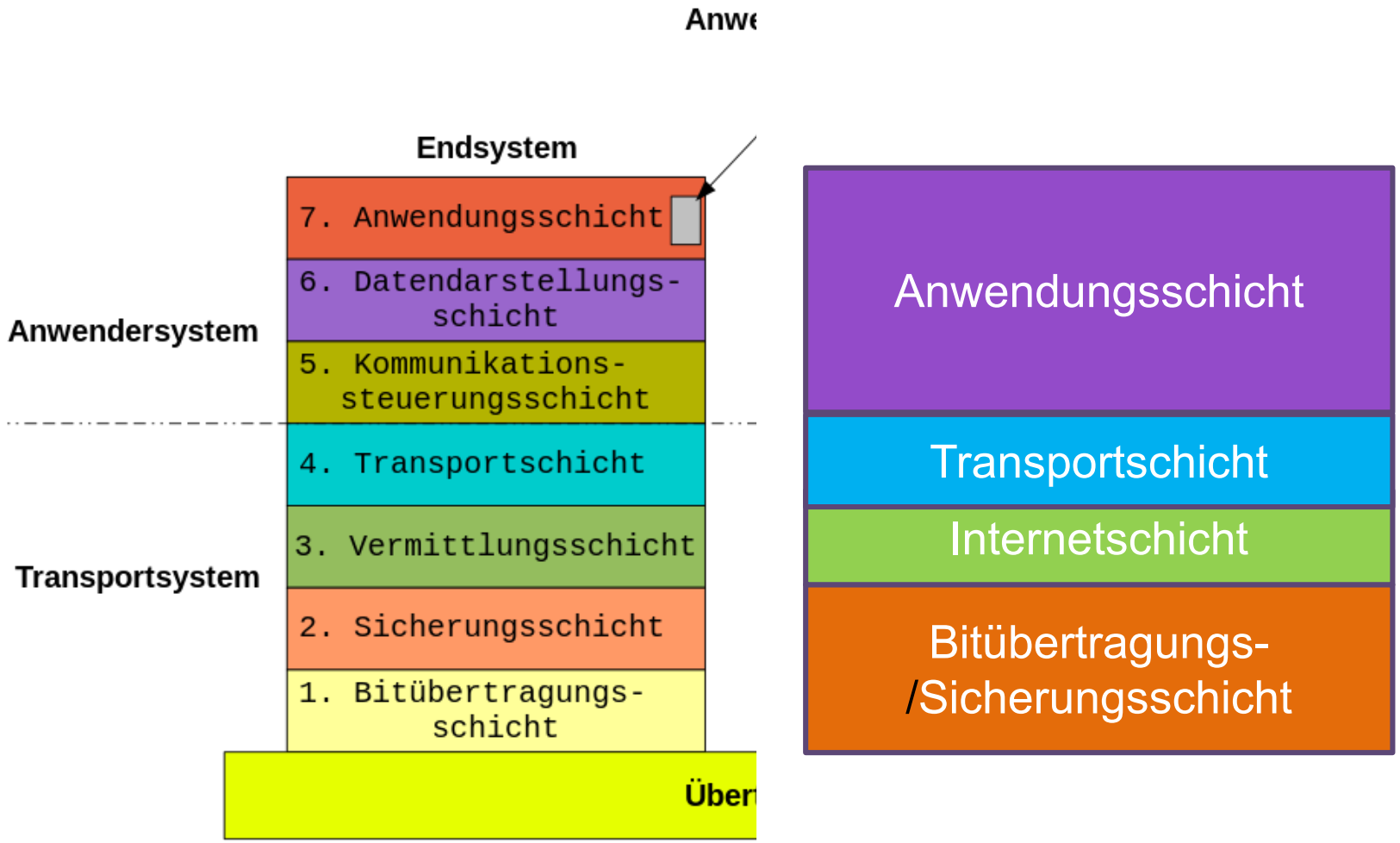
ISO/OSI-REFERENZMODELL

Anwendungsschicht
z.B. ftp, smtp, http



Cc-BY-SA https://commons.wikimedia.org/wiki/File:ISO-OSI-7-Schichten-Modell(in_Deutsch).svg

PRAKTISCH REALISIERT: TCP/IP-MODELL



ÜBERSICHT

Was bisher geschah

Übersicht: Ansätze zur Zerlegung von Problemen

Beispielansätze im Detail

Who‘s Who

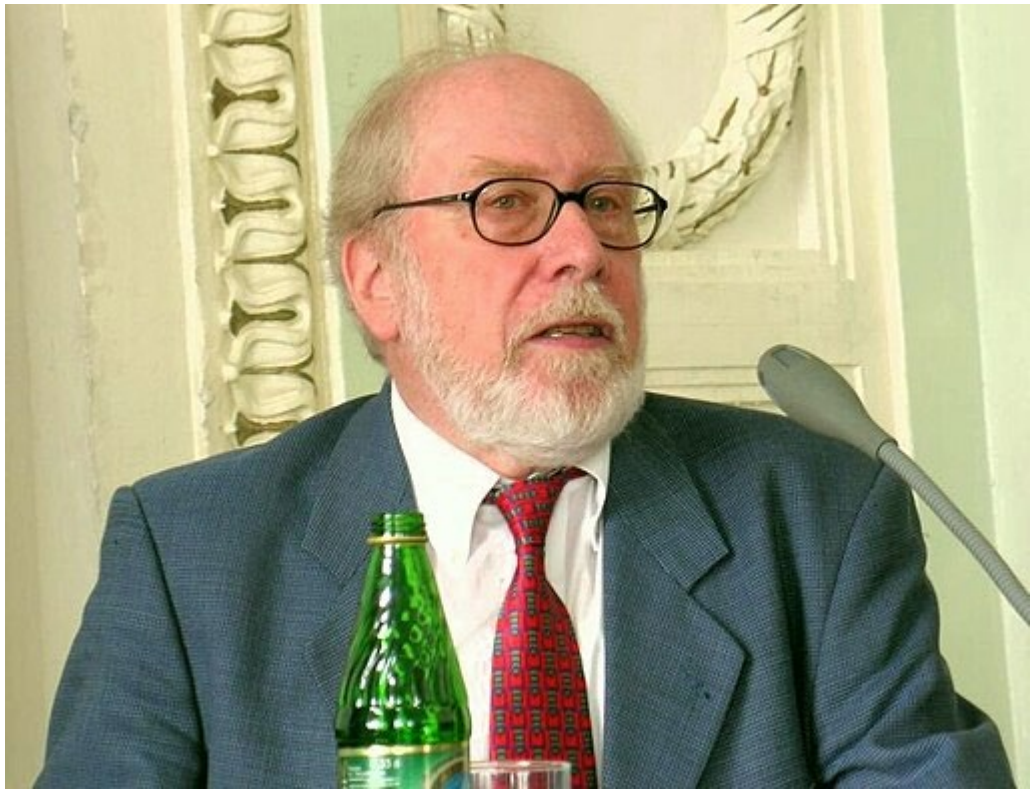
Beispielansätze im Detail

Zusammenfassung und Ausblick

NIKLAUS WIRTH (1934-2024; SCHWEIZ)

Studium in Zürich und Kanada
Promotion University of California at Berkeley (UCB)
Assistenzprofessuren in Stanford und an der U Zürich
1968 – 99 Professur an der ETH

(Mit-)entwicklung mehrerer Programmiersprachen: Algol 68, Modula-2, Oberon



wikimedia

NIKLAUS WIRTH (1934-2024; SCHWEIZ)

Autor mehrerer (erfolgreicher) Lehrbücher

1984: Turing Award



NIKLAUS WIRTH - QUOTES

- **Software gets slower faster than hardware gets faster.**
  - „Wirth‘s Law“
- **In our profession, precision and perfection are not a dispensable luxury, but a simple necessity.**

NIKLAUS WIRTH - QUOTES

- **Whereas Europeans generally pronounce his name the right way ('Nick-louse Veert'), Americans invariably mangle it into 'Nickel's Worth.' This is to say that Europeans call him by name, but Americans call him by value.**
  - Introduction by Adriaan van Wijngaarden at the IFIP Congress (1965).

ÜBERSICHT

Was bisher geschah

Übersicht: Ansätze zur Zerlegung von Problemen

Beispielansätze im Detail

Who‘s Who

Beispielansätze im Detail

Zusammenfassung und Ausblick

FUNKTIONALE MODULARISIERUNG

FUNKTIONALE MODULARISIERUNG

Ähnliche Grundidee wie bei der hierarchischen Zerlegung aber größere Flexibilität

Zerlegung entlang von Aufgaben in Module

Module verstecken, wie sie die Aufgabe lösen

Funktionalität der Module kann über eine Schnittstelle (und nur darüber) genutzt werden.

Interessante Frage: Wie sieht eine gute Zerlegung aus?

Dazu auf den folgenden Folien ein Beispiel von Lutz Prechelt, FU Berlin. (Vielen Dank!)

WAS WIR SCHON WISSEN:

1. **Wichtigster Aspekt ist die Zerlegung in Module:**
   - Ein Modul braucht eine klare **Zuständigkeit** und wird beschrieben über seine **Schnittstelle**
   - Die Schnittstelle verbirgt ein oder mehrere **Geheimnisse** des Moduls
   - Die Geheimnisse spiegeln **Entwurfsentscheidungen** wider, die sich ändern könnten

2. **Ein gut modularisiertes System ist einfach zu verstehen und Änderungen beschränken sich meist auf ein Modul**

WAS WIR NICHT WISSEN: WIE FINDET MAN EINE GUTE MODULARISIERUNG?

Es gibt nirgends eine wirkliche Anleitung, wie man das anstellt

Sondern nur bekannte Kriterien zur Beurteilung des Resultats

Deshalb studieren wir das Problem an einem Fallbeispiel

- KWIC – Key Word in Context

Achtung:

- Das Beispiel ist so einfach, dass die verwendete Modularisierung vielleicht stellenweise übertrieben erscheint
- Bitte stellen Sie sich dann vor, das Problem wäre viel komplexer und übertragen Sie die Überlegungen analog

KWIC: KEY WORD IN CONTEXT



Problemstellung: Suche von Büchern in einer Bibliothek nach Titelstichwort – aber ohne Computer, nur auf ausgedruckten Listen

Eingabe: Liste von Buchtiteln, z.B.

- Java Eclipse Manual
- Eclipse Pocket Guide
- Definitive NetBeans Guide

- Ausgabe:
  "Key Word in Context"-Index
  - Definitive NetBeans Guide
  - Eclipse Manual. Java
  - Eclipse Pocket Guide
  - Guide. Definitive NetBeans
  - Guide. Eclipse Pocket
  - Java Eclipse Manual
  - Manual. Java Eclipse
  - NetBeans Guide. Definitive
  - Pocket Guide. Eclipse

KWIC: SPEZIFIKATION

Eingabe: Eine Menge von Buchtiteln

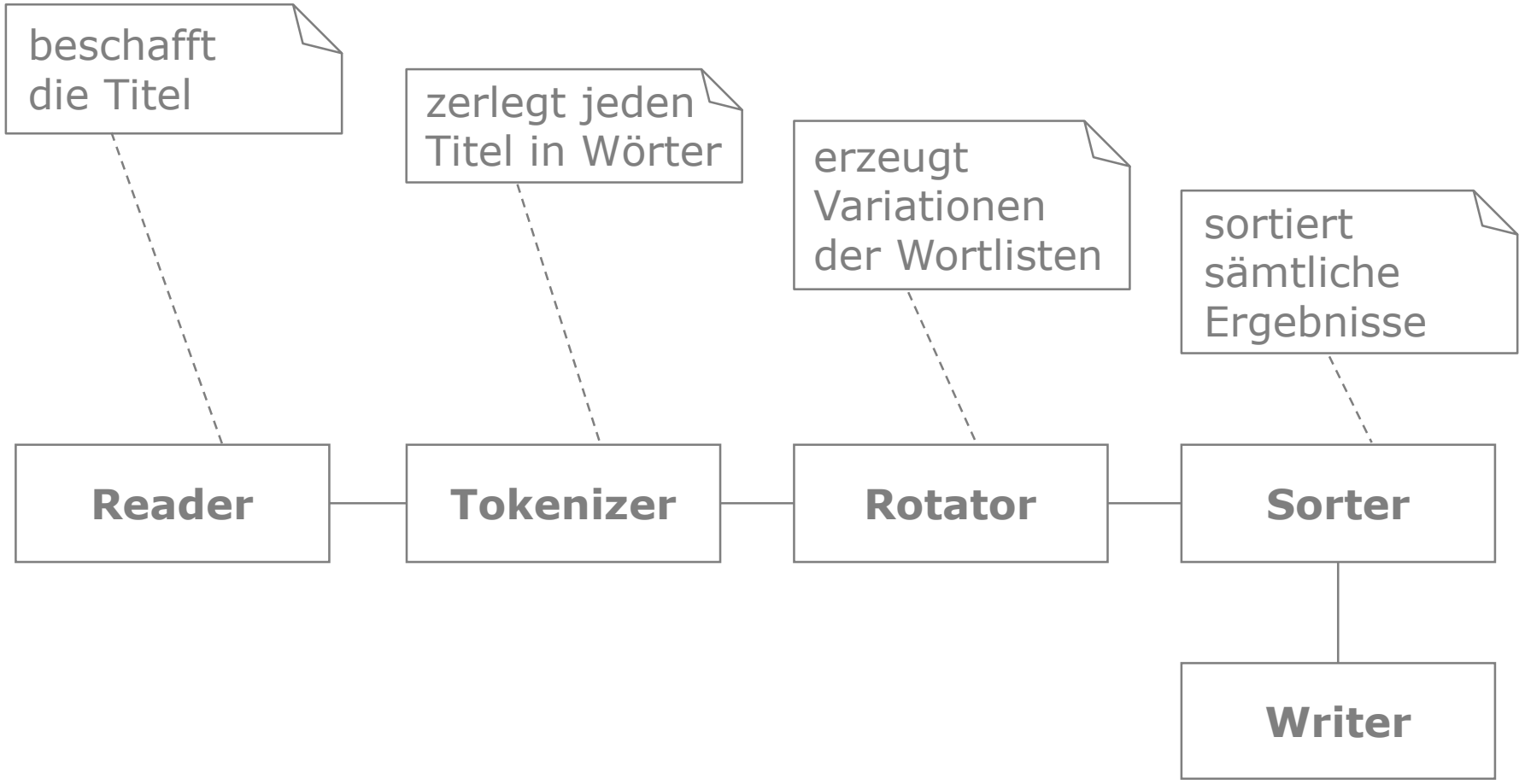
**Erzeugt zu jedem Titel mit *N* Wörtern *N* Ergebnisse, indem so oft das erste Wort ans Ende gestellt wird,
bis jedes Wort einmal vorn gestanden hat**

Diese Ergebnisse ("Varianten") werden für alle Titel gesammelt und die Gesamtmenge sortiert

Ausgabe: Die sortierte Variantenliste

KWIC: MODULARISIERUNG

Ein plausibler Vorschlag (Pipe-and-Filter-Architektur)



KWIC 1:
WIE GUT IST DIE MODULARISIERUNG?

Für wie gut halten Sie diese Modularisierung?

1. **Perfekt**
2. **Gut, ohne erhebliche Schwächen**
3. **OK, es ginge besser, aber sie ist voll brauchbar**
4. **Ausreichend: Funktionsfähig, aber sehr ungünstig aufgeteilt**
5. **Unsinnig**

Wie sicher sind Sie sich in diesem Urteil?

1 Ganz sicher

2 Ich glaube, es stimmt

4 Ich hoffe, dass es stimmt

5 Gar nicht, es ist geraten

KWIC: GEHEIMNISSE DER MODULE

Die Module verbergen relevante Entwurfsentscheidungen:

Reader

- Datenquelle für Titelliste (z.B. Datei, GUI) samt Einleseverfahren

Tokenizer

- Was ist ein Wort?

Rotator

- Datenstruktur und Algorithmus zur effiz. Darstellung d. Varianten
- Behandlung des Titelanfangs im Varianteninnern

Sorter

- Datenstruktur und Algorithmus zum Sortieren
- Sortierordnung

Writer

- Ausgabekanal
- Formataufbereitung

"DAS IST DOCH TRIVIAL!" ?

Diese Geheimnisse sind nicht so banal, wie Sie vielleicht denken

z.B. "Was ist ein Wort" beim *Tokenizer*

- Vorschläge für die Definition?

Ist Ihre Definition für folgende Buchtitel angemessen?

- "Beitrag zur Populationsgenetik der sauren Erythrocytenphosphatase-acP-EC 3.1.3.2 unter besonderer Berücksichtigung des reinerbigen Typus C" (1980)
- "Lepton-Hadron-Korrelationen in (2+1)-Jet-Produktion in tief-inelastischer Elektron-Proton-Streuung zur $O(\alpha_s^2)$" (1992)
- "Die molekulare Wirkung von 2,4,5- und 2,4,6-Trichlorphenol auf Eukaryontenzellen" (1990)

DAS GEHEIMNIS EINES GUTEN *TOKENIZER*

Ein wirklich guter *Tokenizer* würde

- erkennen, um welches Fachgebiet es sich handelt,
- gebietsspezifische Regeln für die Wortbegrenzung verwenden
- und evtl. sogar Wörter anders zusammenbauen!
    - z.B. aus "2,4,5- und 2,4,6-Trichlorphenol"
      mache "2,4,5-Trichlorphenol und 2,4,6-Trichlorphenol"

Er müsste also mehrere fachspezifische Expertensysteme enthalten!

DAS GEHEIMNIS EINES GUTEN *SORTER*

Auch die Sortierordnung ist nicht selbstverständlich:

Wie sortiert man ä æ å ?

- Für Deutsche: ä=ae æ=ae å=a
- Für Amerikaner_innen: ä=a æ=ae å=a
- Für Skandinavier_innen: ä>z æ>z å>z

Wie sortiert man ß ?

- Für Deutsche: ß=ss
- Für Amerikaner_innen?
- Für Skandinavier_innen?

Wie sortiert man ð þ ł ŋ ?

- Alles europäische Buchstaben!

Wie sortiert man asiatische Ideogrammschriften?

DAS GEHEIMNIS EINES GUTEN *WRITER*

Die Ausgabe der Liste kann ganz verschieden sein:

Im einfachen Fall eine schlichte Textdatei

- nur die rohen Ergebnisse

Aber vielleicht wollen wir ja ein schönes PDF?

- Dann muss das Modul ziemlich viel "wissen". Teilaufgaben:
- PDF-Dateiformat
- Seitenumbruch (Seitengrößen, Fonthöhen etc.)
- Kopfzeilen, Fußzeilen, Seitennummern etc.
- Inhaltsverzeichnis (per Anfangsbuchstabe)

KWIC 1:
WIE GUT IST DIE MODULARISIERUNG?

Für wie gut halten Sie diese Modularisierung?

1 Perfekt

2 Gut, ohne erhebliche Schwächen

3 OK, es ginge besser, aber sie ist voll brauchbar

4 Ausreichend: Funktionsfähig, aber sehr ungünstig aufgeteilt

5 Unsinnig

Wie sicher sind Sie sich in diesem Urteil?

1 Ganz sicher

2 Ich glaube, es stimmt

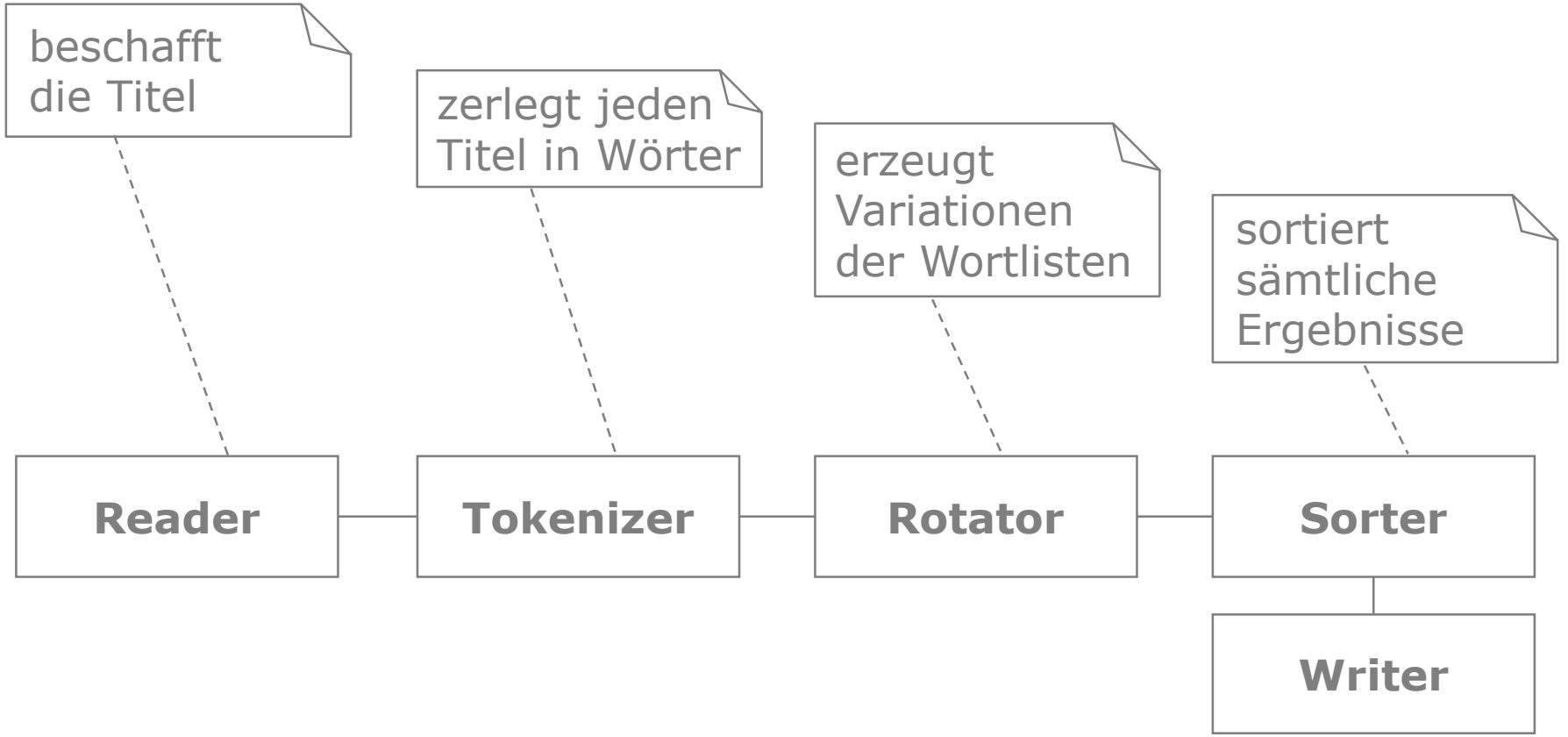
4 Ich hoffe, dass es stimmt

5 Gar nicht, es ist geraten

ERINNERUNG: KWIC MODULARISIERUNG

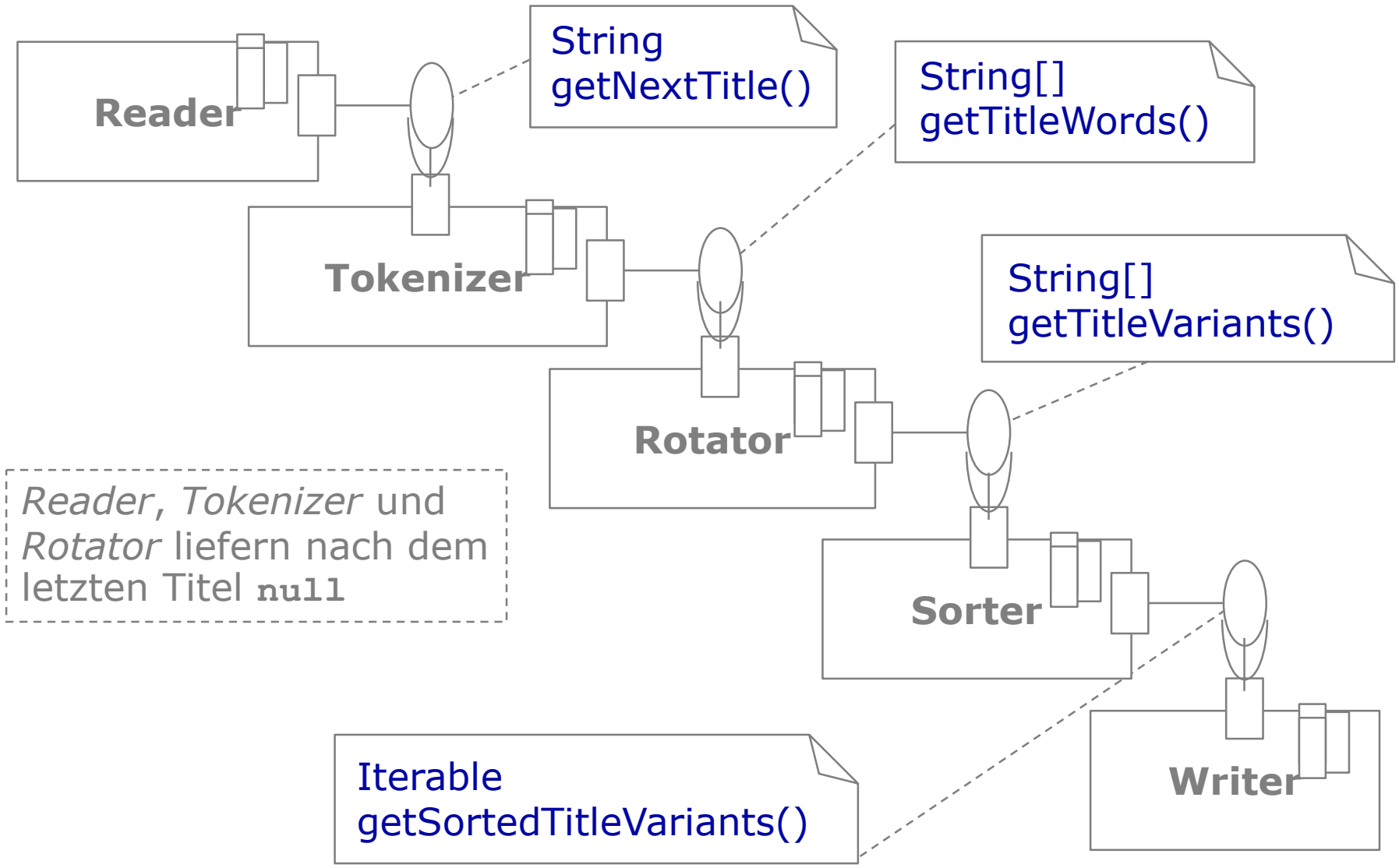
Diese Darstellung ist reichlich informell und vage

- Besser wäre eine richtiges Komponentendiagramm
- mit Angabe der Schnittstellen



Lutz Prechelt, prechelt@inf.fu-berlin.de

KWIC
ALS UML-KOMPONENTENDIAGRAMM



KWIC: HAUPTPROGRAMM

Das Hauptprogramm (steht in *Writer*) sieht bei dieser Modularisierung sinngemäß etwa so aus:

```
Reader reader       = new FileReader(args[0]);
Tokenizer tokenizer = new Tokenizer(reader);
Rotator rotator     = new Rotator(tokenizer);
Sorter sorter       = new Sorter(rotator);
Writer writer       = new Writer(sorter);
writer.writeKWIClist();
```

KWIC: MÄNGEL DIESES ENTWURFS

1. Die Module sind unnötig fest aneinander gekoppelt

- Das erkennt man am Hauptprogramm:
- Wollte man noch einen Zwischenschritt einfügen, müssten bestehende Schnittstellen geändert werden (Konstruktoren)
    - d.h. die Art und Reihenfolge der Schritte ist nicht Geheimnis irgendeines Moduls

2. Die Schnittstellen der Module sind hochspezialisiert für genau diese Anwendung

- Obwohl man z.B. einen *Tokenizer* ja auch woanders wiederverwenden könnte
- Die Module sollten besser alle von einem Treiber aus aufgerufen werden, anstatt sich gegenseitig als Kette aufzurufen
- Dann würden die Schnittstellen allgemeiner und die Module wären leichter verständlich und besser wiederzuverwenden
    - Denn dann wären Art und Reihenfolge der Schritte das Geheimnis dieses Treibermoduls

KWIC 1:
WIE GUT IST DIE MODULARISIERUNG?

Für wie gut halten Sie diese Modularisierung?

1. **Perfekt**
2. **Gut, ohne erhebliche Schwächen**
3. **OK, es ginge besser, aber sie ist voll brauchbar**
4. **Ausreichend: Funktionsfähig, aber sehr ungünstig aufgeteilt**
5. **Unsinnig**

Wie sicher sind Sie sich in diesem Urteil?

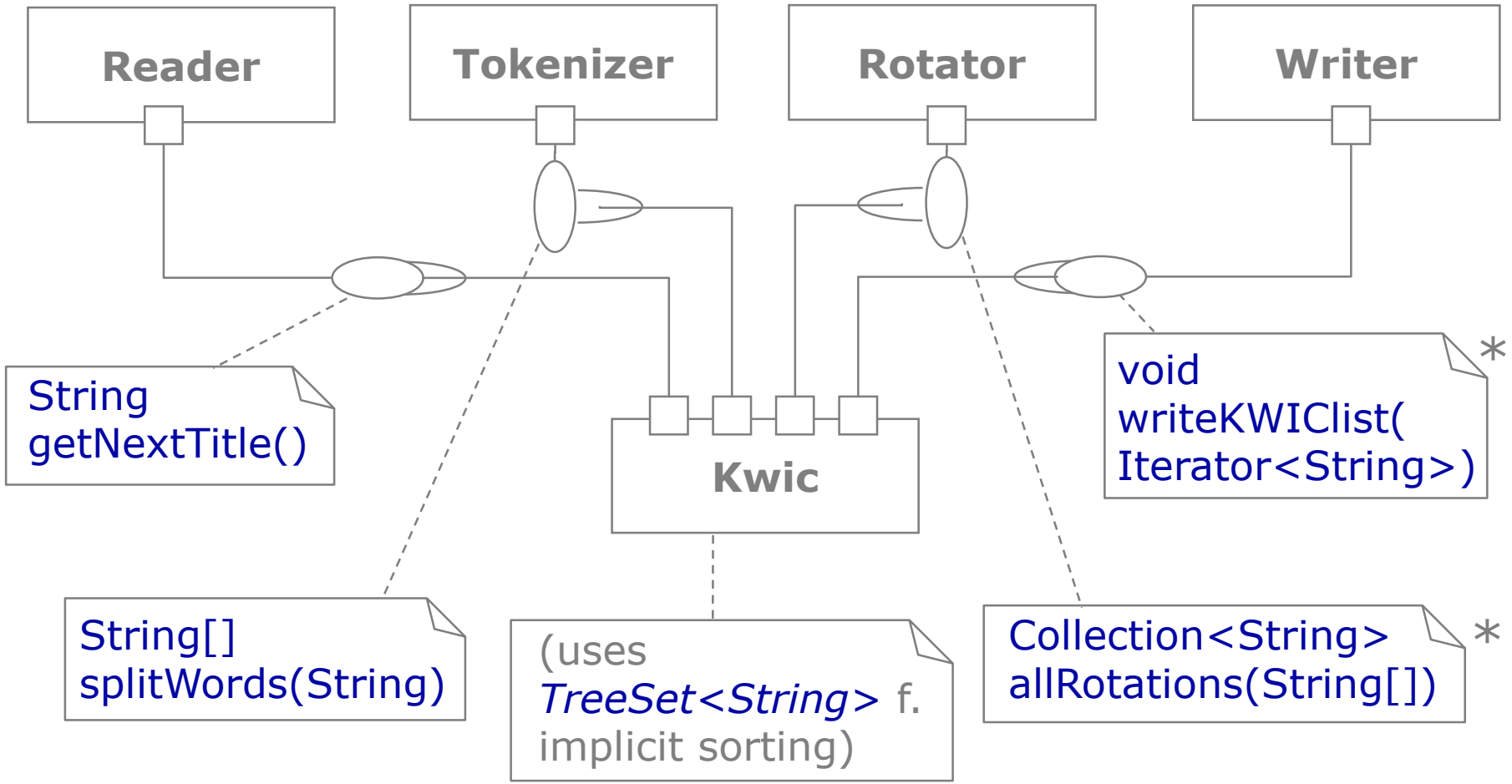
1 Ganz sicher

2 Ich glaube, es stimmt

4 Ich hoffe, dass es stimmt

5 Gar nicht, es ist geraten

KWIC VERSION 2: MIT TREIBER



KWIC 2: (*) ANMERKUNGEN ZU DEN SCHNITTSTELLEN

Natürlich gilt auch hier:

Die Signatur ist nicht die Schnittstelle!

Insbesondere muss genau der Inhalt der *Collection* und des *Iterators* definiert werden

- In Java sind sie als *Collection*<*String*> und *Iterator*<*String*> definiert – das hilft schon mal *ein wenig*
- Aber *allRotations()* sollte z.B. beschreiben:
  "A list of N words will result in N variants as follows:
  - Variant 1 is the original list.
  - Variant i+1 is produced from variant i by removing the first word and appending it to the end."

KWIC VERSION 2: VERBESSERUNGEN

1. *Tokenizer* und *Rotator* haben jetzt "natürliche" Schnittstellen

- Die enge Kopplung an den Rest des Systems ist verschwunden
- Dadurch sind die Module jetzt wiederverwendbar

2. Das System wird offener für Erweiterungen

- Einfach in Modul *Kwic* ergänzen

3. *Sorter* ist nicht mehr nötig

- Statt dessen benutzt *Kwic* direkt die entsprechenden Klassen aus der Bibliothek:
  - java.util.**TreeSet** sammelt Varianten und sortiert sie
  - java.text.**RuleBasedCollator** legt eine Sortierreihenfolge fest

KWIC 2:
WIE GUT IST DIE MODULARISIERUNG?

Für wie gut halten Sie diese Modularisierung?

1. **Perfekt**
2. **Gut, ohne erhebliche Schwächen**
3. **OK, es ginge besser, aber sie ist voll brauchbar**
4. **Ausreichend: Funktionsfähig, aber sehr ungünstig aufgeteilt**
5. **Unsinnig**

Wie sicher sind Sie sich in diesem Urteil?

1 Ganz sicher

2 Ich glaube, es stimmt

4 Ich hoffe, dass es stimmt

5 Gar nicht, es ist geraten

KWIC 2: GRENZEN DIESES ENTWURFS

Dieser Entwurf ist gut, solange der komplette KWIC-Index bequem in den Hauptspeicher passt

Für diese Anwendung (KWIC) ist das akzeptabel, aber in anderen Fällen wäre es gut, eine Architektur zu haben, die

- entweder den Speicheraufwand stark reduziert
- oder erlaubt, die Daten im Hintergrundspeicher zu halten (Platte)

KWIC 2: SPEICHERBEDARF

KWIC-Listen enthalten offensichtlich sehr viel Redundanz

- immer wieder die gleichen Wörter
- immer wieder die gleichen Wortfolgen (nur verschoben)

Bei langen Titeln wird der Speicherbedarf recht groß:

- *"Untersuchungen über Supravitalfärbungen an Bullenspermien im Vergleich zu deren Bewegungsverhalten im Nativpräparat unter besonderer Berücksichtigung verschiedener Samenverdünnungen und -aufbewahrungen"* (1959)
- *"Ermittlung und Wichtung der Merkmale zur Analyse und zum Vergleich von Milchproduktionsanlagen in Zusammenarbeit mit 15 Leitern der Tierproduktion von landwirtschaftlichen Betrieben der Deutschen Demokratischen Republik (VEG und LPG)"* (1972)
    - 29 Wörter. Angenommen, wir hätten 1 Million solcher Titel:
- ohne Redundanz: 230 Zeichen Rohdaten ➔ 230 MB
- mit Redundanz: 6670 Zeichen Variantendaten ➔ 6,7 GB

KWIC 2:
WIE GUT IST DIE MODULARISIERUNG?

Für wie gut halten Sie diese Modularisierung?

1 Perfekt

2 Gut, ohne erhebliche Schwächen

3 OK, es ginge besser, aber sie ist voll brauchbar

4 Ausreichend: Funktionsfähig, aber sehr ungünstig aufgeteilt

5 Unsinnig

Wie sicher sind Sie sich in diesem Urteil?

1 Ganz sicher

2 Ich glaube, es stimmt

4 Ich hoffe, dass es stimmt

5 Gar nicht, es ist geraten

KWIC VERSION 3: MIT DATENABSTRAKTION

Speicherersparnis gelingt, wenn man anstelle von *String* für Wörter und Varianten entsprechende abstrakte Datentypen benutzt:

- *UString*: Gleiche Strings werden nur einmal angelegt
  - Die meisten Java VMs machen das ohnehin so, aber keine Garantie
- *WordList*: Varianten eines Titels teilen sich ein *UString*-Array

Durch Einführung von *UString* und *WordList* ändert sich am restlichen Programm nur wenig

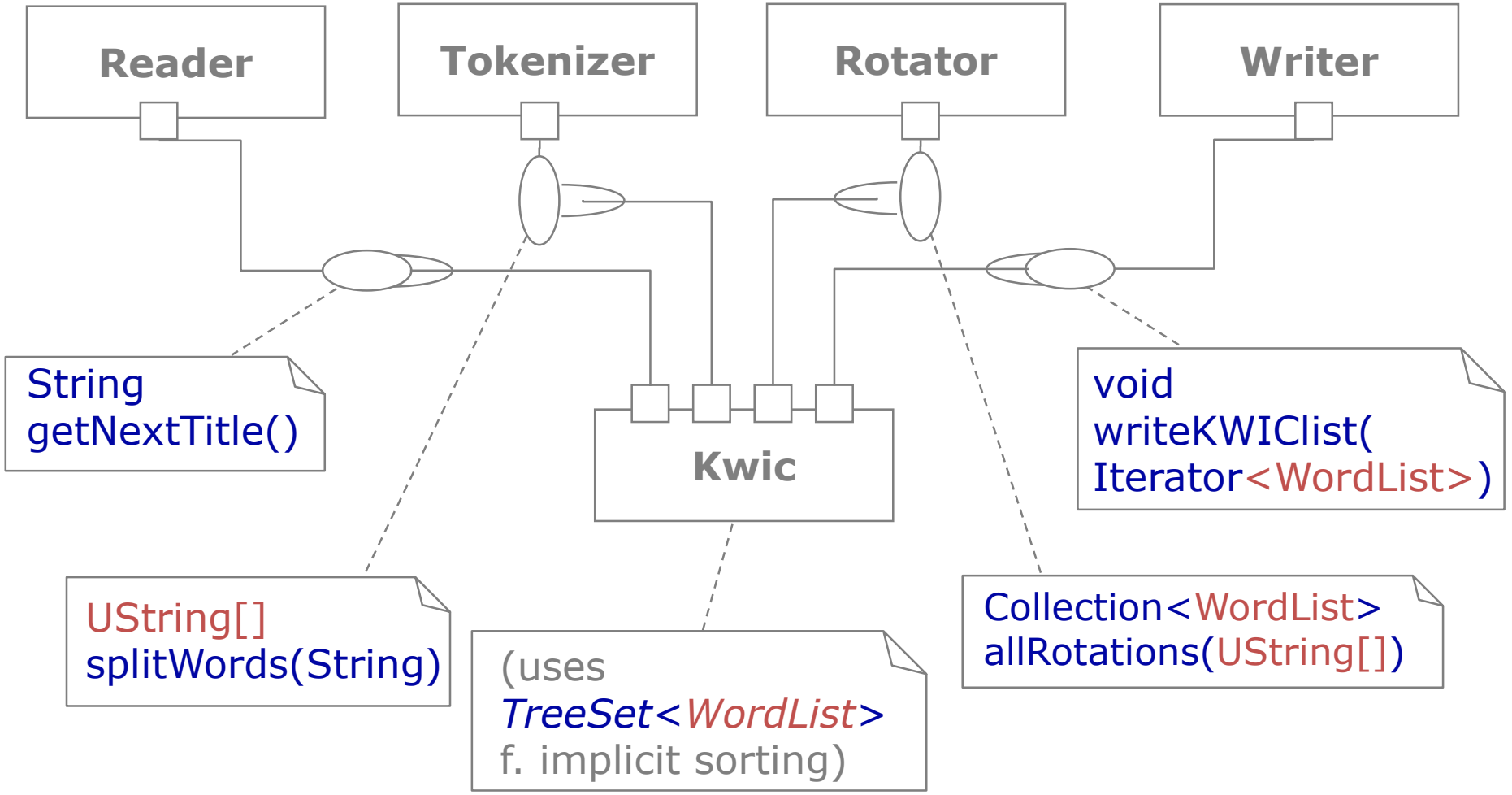
- meist müssen nur Typnamen ersetzt werden

aber wir können nun zwei Geheimnisse nach *WordList* verlegen

- Sortierreihenfolge: static int compare (WordList, WordList)
- Format f. Variante: String toString()

und der Speicherbedarf für große KWIC-Listen sinkt stark

KWIC 3: MIT USTRING UND WORDLIST



Module *UString* und *WordList* sind nicht separat dargestellt

KWIC 3:
WIE GUT IST DIE MODULARISIERUNG?

Für wie gut halten Sie diese Modularisierung?

1. **Perfekt**
2. **Gut, ohne erhebliche Schwächen**
3. **OK, es ginge besser, aber sie ist voll brauchbar**
4. **Ausreichend: Funktionsfähig, aber sehr ungünstig aufgeteilt**
5. **Unsinnig**

Wie sicher sind Sie sich in diesem Urteil?

1 Ganz sicher

2 Ich glaube, es stimmt

4 Ich hoffe, dass es stimmt

5 Gar nicht, es ist geraten

KWIC 1,2,3: ÄNDERUNGSFREUNDLICHKEIT

Jetzt untersuchen wir das Verhalten der drei Entwürfe bei Änderungen am System

Änderung 1: Stoppwortliste

Änderung 2: Wortumbruch

Änderung 3: Automatisches Abkürzen

Änderung 4: Datenhaltung auf Hintergrundspeicher

Lutz Prechelt, prechelt@inf.fu-berlin.de

KWIC ÄNDERUNG 1: EINFÜHRUNG EINER STOPPWORTLISTE

Es gibt in Titeln viele nichtssagende Wörter

- Artikel ("der", "ein", …), Konjunktionen ("und", "weil", …), Pronomen ("die", "dessen",…)

KWIC-Einträge, die mit einem solchen "Stoppwort" beginnen, sollen unterdrückt werden

- Die Stoppwortliste ist das Geheimnis eines eigenen Moduls

Die Änderung sieht in allen drei KWIC-Versionen gleich aus:

- Realisierung eines Stoppworttests in einem Extramodul
- Aufruf des Tests im *Rotator*
    - Oder bei KWIC 1: zur Erhaltung der pipe-and-filter-Architektur als eigene Filterstufe realisiert → verlangt Schnittstellenänderung

Lutz Prechelt, prechelt@inf.fu-berlin.de

KWIC ÄNDERUNG 2: WORTUMBRUCH IN DER AUSGABE

Lange Titel passen in der Ausgabe nicht auf eine Zeile

***Writer* soll einen geeigneten Wortumbruch vornehmen**

- Das genaue Verfahren wird sein zusätzliches Geheimnis
  - Man könnte auch ein separates Modul dafür einführen

KWIC 1, KWIC 2:

- *Writer* muss dazu zuerst die fertig formatierten Variantenstrings wieder in Wörter zerlegen
- Das ist ungünstig, denn wir haben ja zuvor mal Wörter gehabt

KWIC 3:

- Die Varianten kommen als *WordList*-Objekte an
- Der Wortumbruch kann also bei der Formatierung recht einfach durchgeführt werden

Lutz Prechelt, prechelt@inf.fu-berlin.de

KWIC-ÄNDERUNG 3: AUTOMATISCHES ABKÜRZEN

Es wäre sinnvoll, bestimmte häufige Wörter automatisch bei der KWIC-Erzeugung durch eine Abkürzung zu ersetzen ("Konferenz"→"Konf.", "international"→"int'l.",…)

Dafür soll ein Abkürzmodul *Compactor* ergänzt werden

- Aufruf zwischen *Tokenizer* und *Rotator*

KWIC 1:

- Schnittstelle von *Rotator* ändert sich
- Hauptprogramm ändert sich

KWIC 2, KWIC 3:

- Modulschnittstellen bleiben gleich
- Nur Hauptprogramm ändert sich

KWIC ÄNDERUNG 4: DATENHALTUNG AUF HINTERGRUNDSPEICHER

Bei riesigen Mengen von Titeln kann der KWIC-Index nicht mehr im Hauptspeicher gehalten werden

- selbst mit KWIC 3 vielleicht nicht

Die ganze Datenhaltung der Titelwörter soll deshalb in einem Datenbankmanagementsystem (DBMS) erfolgen

KWIC 1:

- Totale Umstrukturierung von *Rotator*, *Sorter* und *Writer* nötig
  - und vieler weiterer Module, sollten sie existieren
- Resultierendes System wird sehr kompliziert

KWIC 2:

- Totale Umstrukturierung von *Rotator*, *Writer* und *Kwic*
- Resultierendes System wird sehr kompliziert

KWIC 3:

- Änderungen (im Prinzip) beschränkt auf *UString* und *WordList*
  - Egal wie viele andere Module existieren

ÄNDERUNGSFREUNDLICHKEIT

- Die Änderungsfreundlichkeit der drei Modularisierungen im direkten Vergleich:

Änderung	KWIC 1	KWIC 2	KWIC 3
1. Stoppwortliste	(+)	+	+
2. Wortumbruch	-	-	+
3. Autom. Abkürzen	-	+	+
4. Hintergrundspeicher	-	-	+

KWIC 3:
WIE GUT IST DIE MODULARISIERUNG?

Für wie gut halten Sie diese Modularisierung?

1 Perfekt

2 Gut, ohne erhebliche Schwächen

3 OK, es ginge besser, aber sie ist voll brauchbar

4 Ausreichend: Funktionsfähig, aber sehr ungünstig aufgeteilt

5 Unsinnig

Wie sicher sind Sie sich in diesem Urteil?

1 Ganz sicher

2 Ich glaube, es stimmt

4 Ich hoffe, dass es stimmt

5 Gar nicht, es ist geraten

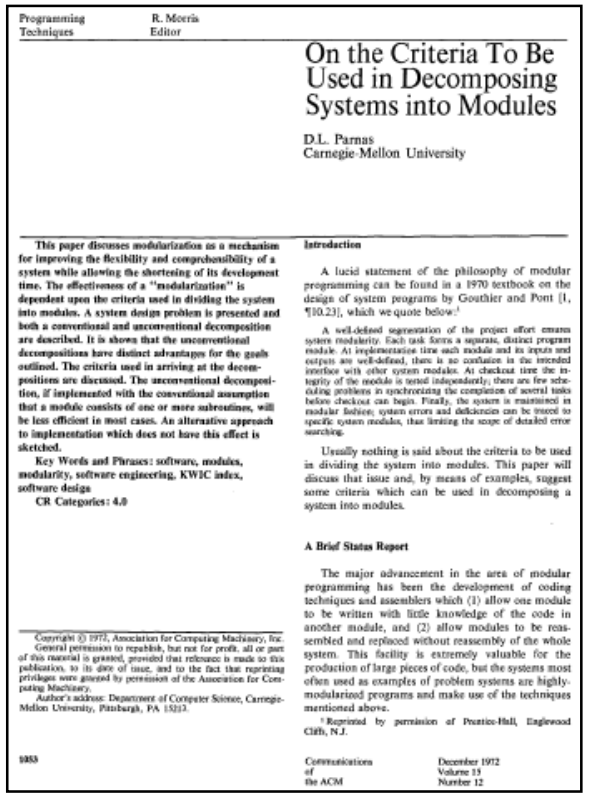
KWIC: QUELLE

Das KWIC-Beispiel ist angelehnt an einen klassischen Artikel:

- David Parnas: **"On the criteria to be used in decomposing systems into modules"**, Communications of the ACM 15(12):1053-1058, December 1972

In diesem Artikel wurde erstmals das Prinzip der Verbergung von Entwurfs-entscheidungen ("information hiding") explizit formuliert.

- Auch damals kannten viele Programmierer bereits das Konzept von Modulen, zerteilten ihr Programm aber recht willkürlich oder nach anderen Kriterien
- Auch heute gilt noch: Viele Entwerfer verbergen nicht die wichtigen veränderlichen Entwurfsentscheidungen!
  - OO lenkt davon nämlich eher ab



Programming Techniques — R. Morris, Editor

On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas
Carnegie-Mellon University

This paper discusses modularization as a mechanism for improving the flexibility and comprehensibility of a system while allowing the shortening of its development time. The effectiveness of a "modularization" is dependent upon the criteria used in dividing the system into modules. A system design problem is presented and both a conventional and unconventional decomposition are described. It is shown that the unconventional decompositions have distinct advantages for the goals outlined. The criteria used in arriving at the decompositions are discussed. The unconventional decomposition, if implemented with the conventional assumption that a module consists of one or more subroutines, will be less efficient in most cases. An alternative approach to implementation which does not have this effect is sketched.

Key Words and Phrases: software, modules, modularity, software engineering, KWIC index, software design
CR Categories: 4.0

Copyright © 1972, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted, provided that reference is made to this publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

Author's address: Department of Computer Science, Carnegie-Mellon University, Pittsburgh, PA 15213.

Introduction

A lucid statement of the philosophy of modular programming can be found in a 1970 textbook on the design of system programs by Gouthier and Pont [1, ¶10.23], which we quote below:[1]

A well-defined segmentation of the project effort ensures system modularity. Each task forms a separate, distinct program module. At implementation time each module and its inputs and outputs are well-defined, there is no confusion in the intended interface with other system modules. At checkout time the integrity of the module is tested independently; there are few scheduling problems in synchronizing the completion of several tasks before checkout can begin. Finally, the system is maintained in modular fashion; system errors and deficiencies can be traced to specific system modules, thus limiting the scope of detailed error searching.

Usually nothing is said about the criteria to be used in dividing the system into modules. This paper will discuss that issue and, by means of examples, suggest some criteria which can be used in decomposing a system into modules.

A Brief Status Report

The major advancement in the area of modular programming has been the development of coding techniques and assemblers which (1) allow one module to be written with little knowledge of the code in another module, and (2) allow modules to be reassembled and replaced without reassembly of the whole system. This facility is extremely valuable for the production of large pieces of code, but the systems most often used as examples of problem systems are highly-modularized programs and make use of the techniques mentioned above.

[1] Reprinted by permission of Prentice-Hall, Englewood Cliffs, N.J.

FAZIT: ERKENNTNISSE

Zweifel sind beim Entwurf hilfreich!

1. **Kisten-und-Pfeile-Bildchen reichen nicht aus, um einen Entw beschreiben oder zu verstehen**
2. **Eine gute Modularisierung zu finden ist schwierig**
   - Viele Aspekte spielen eine Rolle
   - Wenn man die entscheidenden Änderungen nicht vorhersehen kann, gelingt es allenfalls mit Glück
     - Deswegen sind Standard-Architekturen so nützlich:
       Da ist das Glück quasi fest eingebaut
3. **Sinnvoll aussehende Modularisierungen sind oft nicht so günstig wie sie scheinen (siehe KWIC 1)**
   - Siehe die Meinungsschwankungen in den Umfragen
4. **Die Änderungsfreundlichkeit muss gegen die Entwurfskomplexität abgewogen werden**
   - KWIC 2 ist garantiert besser als KWIC 1
   - Aber ob KWIC 3 einen Fortschritt darstellt, ist *nicht* völlig klar, denn es ist aufwändiger als KWIC 2

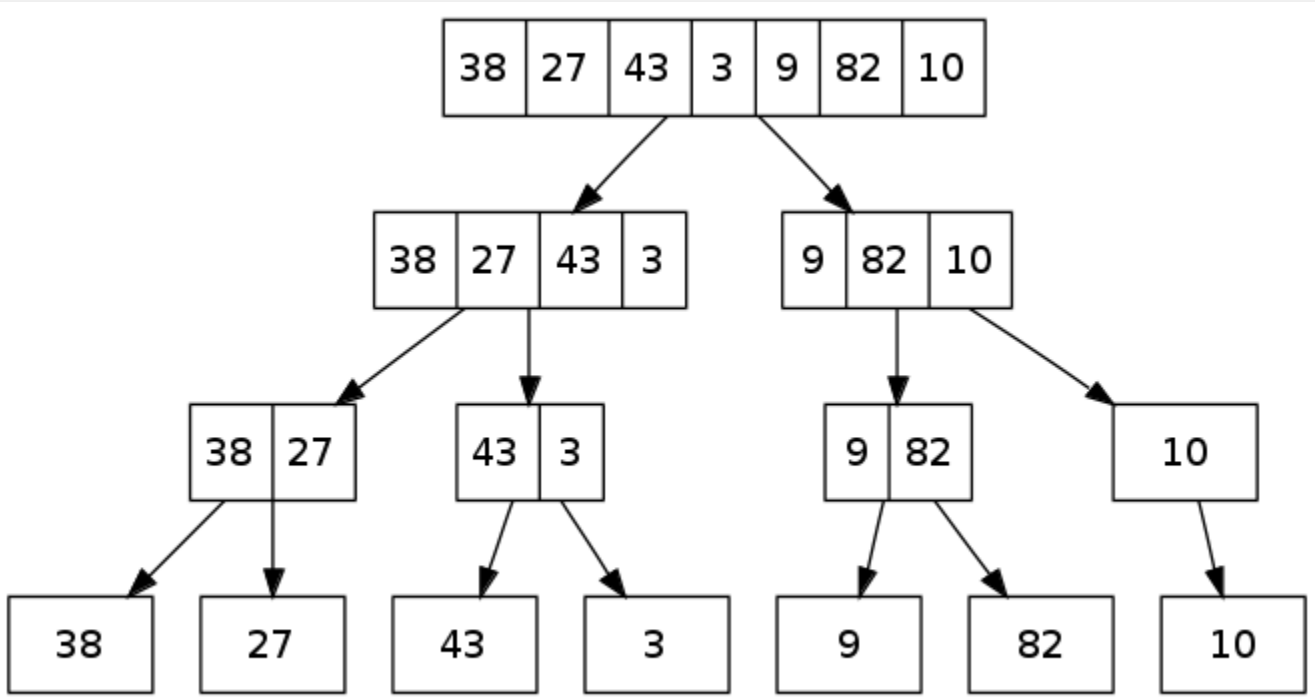
REKURSIVE ZERLEGUNG

REKURSIVE ZERLEGUNG

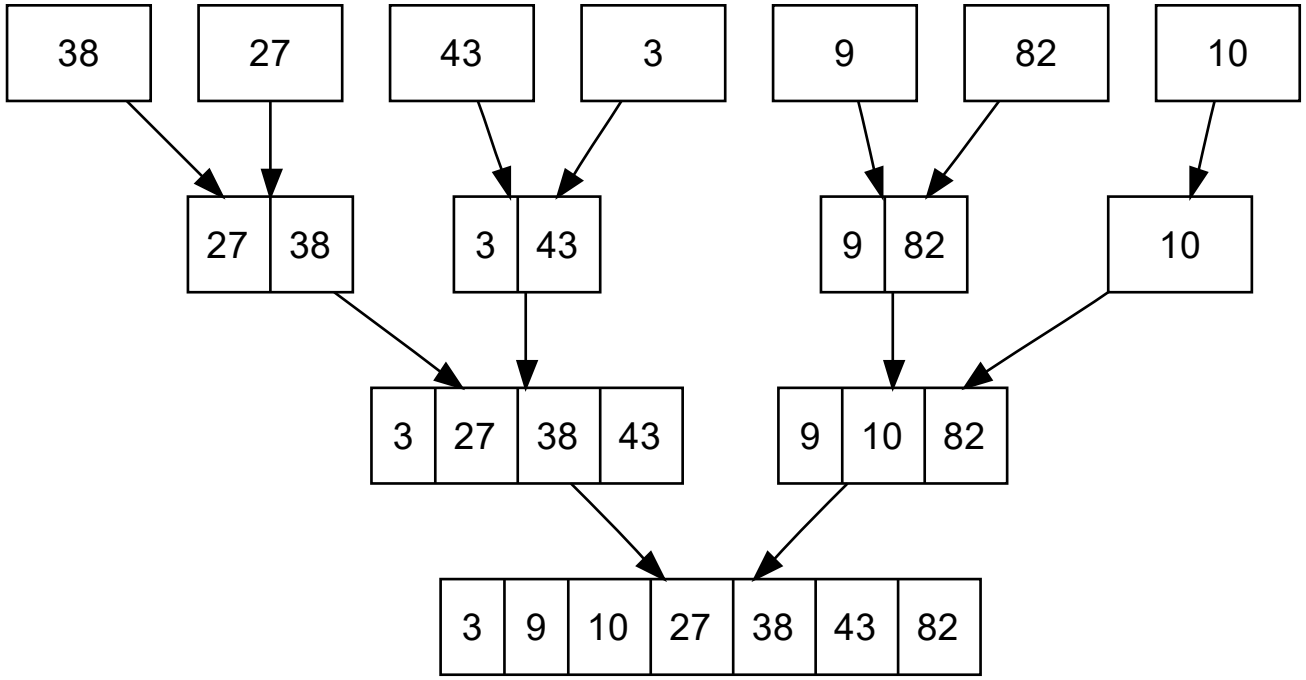
= Zerlegung eines Problems in kleinere, gleichartige Probleme



Beispiel: Merge Sort



Beispiel: Merge Sort



PARALLELISIERUNG

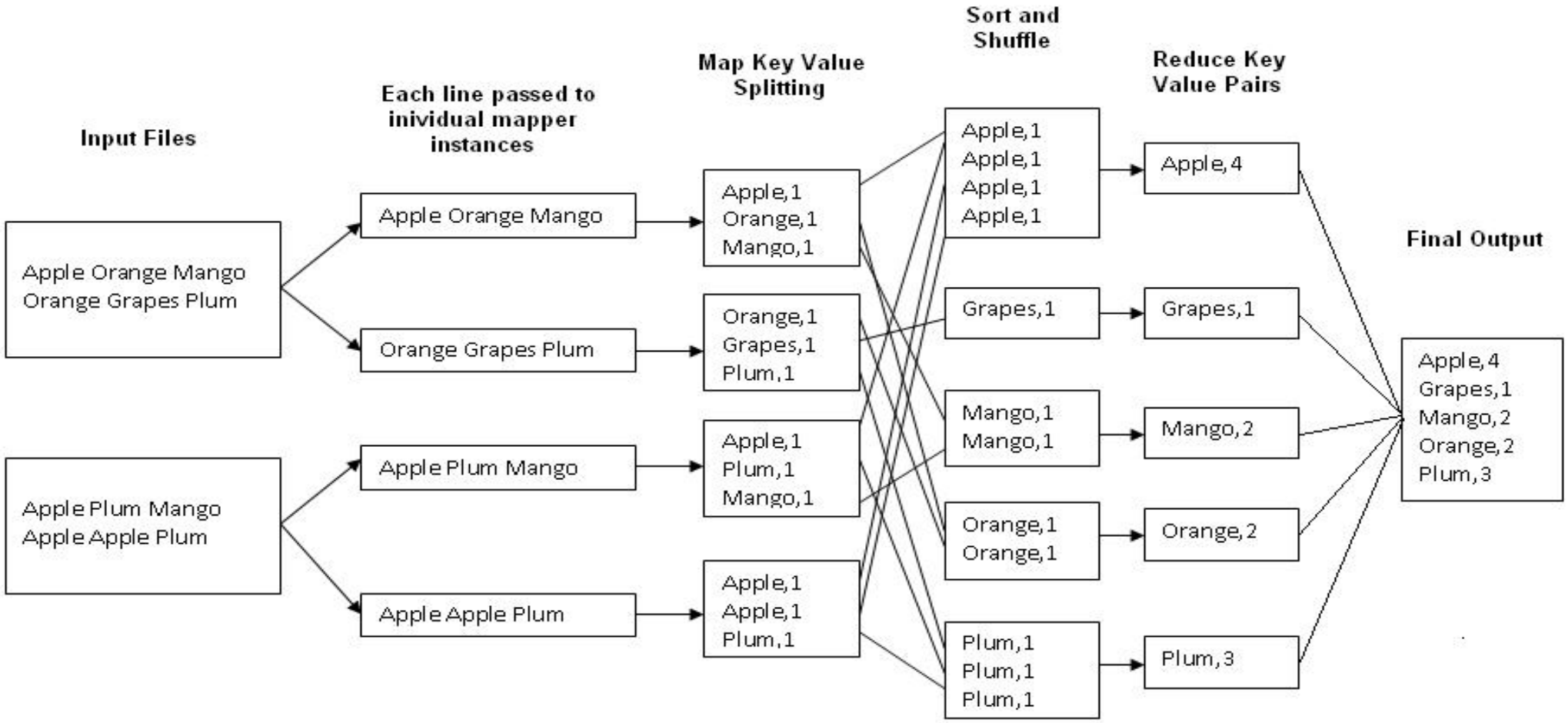
PARALLELISIERUNG

- = gleichartige Teilprobleme gleichzeitig bearbeiten
- Relativ einfach, wenn Probleme so zerlegt werden können, dass die Teilprobleme unabhängig voneinander bearbeitet werden können (Datenparallelität)

Beispiel 2: MapReduce

- von Google eingeführtes Programmiermodell für verteilte Verarbeitung großer Datenmengen auf mehreren Rechnern
- 3 Phasen: **map**, shuffle, **reduce**
- Anwendungsbereiche:
  - Bestimmen von Worthäufigkeiten,
  - verteilte Suche
  - verteilte Lernalgorithmen,
  - ...

Beispiel 2: MapReduce



ÜBERSICHT

Was bisher geschah

Übersicht: Ansätze zur Zerlegung von Problemen

Beispielansätze im Detail

Who‘s Who

Beispielansätze im Detail

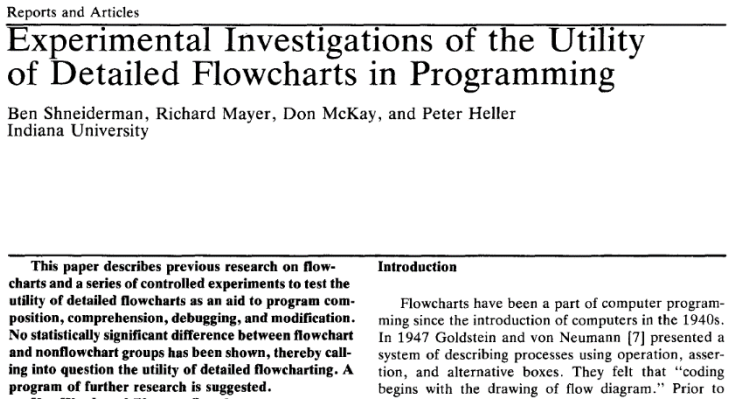
Zusammenfassung und Ausblick

TAKE HOME MESSAGE

- **Problemzerlegung wichtiges Instrument zur Problemlösung**
- **Unterschiedliche Ansätze, z.B.**
  - Modularisierung, Schichtenarchitekturen, Rekursive Zerlegung, Parallelisierung
  - Weitere, die wir nicht angeguckt haben
- **Gute Zerlegung ist nicht trivial!**

TERMIN 3: DATEN UND IHRE REPRÄSENTATION

- Daten und Informationen
- Grundformen der Datenrepräsentation (Zahlen, Text, Bilder)
- Codierung
- Komprimierung



Reports and Articles

Experimental Investigations of the Utility of Detailed Flowcharts in Programming

Ben Shneiderman, Richard Mayer, Don McKay, and Peter Heller
Indiana University

This paper describes previous research on flowcharts and a series of controlled experiments to test the utility of detailed flowcharts as an aid to program composition, comprehension, debugging, and modification. No statistically significant difference between flowchart and nonflowchart groups has been shown, thereby calling into question the utility of detailed flowcharting. A program of further research is suggested.

Introduction

Flowcharts have been a part of computer programming since the introduction of computers in the 1940s. In 1947 Goldstein and von Neumann [7] presented a system of describing processes using operation, assertion, and alternative boxes. They felt that "coding begins with the drawing of flow diagram." Prior to







QUELLENANGABEN

Bilder soweit nicht direkt zitiert: Adobe Stock

Modularisierungsbeispiel: Lutz Prechelt, FU Berlin

Einige Beispiele und Übungsformate mit Hilfe von ChatGPT erstellt

